

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO DỰ ÁN

Chuyên ngành: Công nghệ thông tin

**XÂY DỰNG ỨNG DỤNG DI ĐỘNG QUẢN LÝ DINH
DƯỠNG VÀ LUYỆN TẬP CÁ NHÂN HÓA SMARTNUTRI**

Người thực hiện:	Trịnh Quốc Khánh Nguyễn Đức Huy
Mã học viên/MSSV:	22021204 22021186
Lớp:	K67IT1
Giảng viên hướng dẫn:	ThS Vương Thị Hồng

HÀ NỘI – 2026

Lời cảm ơn

Chúng tôi xin gửi lời cảm ơn chân thành đến **ThS Vương Thị Hồng**, người đã tận tình hướng dẫn, góp ý và động viên nhóm trong suốt quá trình thực hiện đồ án này. Những nhận xét và định hướng của cô là nền tảng quan trọng giúp nhóm hoàn thiện dự án.

Chúng tôi cũng xin cảm ơn quý thầy cô trong Khoa Công nghệ Thông tin, Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã trang bị cho nhóm những kiến thức quý báu trong suốt thời gian học tập, làm cơ sở để nhóm có thể thực hiện được dự án này.

Cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ nhóm trong suốt quá trình học tập và thực hiện đồ án.

Trong quá trình thực hiện, mặc dù đã cố gắng hết sức nhưng không thể tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ quý thầy cô để báo cáo được hoàn thiện hơn.

Chúng tôi xin chân thành cảm ơn.

Hà Nội, 2026

Nhóm sinh viên thực hiện

Lời cam đoan

Chúng tôi xin cam đoan đây là công trình nghiên cứu của nhóm, được thực hiện dưới sự hướng dẫn của **ThS Vương Thị Hồng**.

Các số liệu, kết quả trình bày trong báo cáo này là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Các tài liệu tham khảo, đoạn trích dẫn, hình ảnh và số liệu từ các nguồn khác đều được trích dẫn đầy đủ và ghi rõ nguồn gốc trong danh mục tài liệu tham khảo.

Nhóm xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

Hà Nội, ngày 16 tháng 5 năm 2026

Nhóm sinh viên thực hiện

Tóm tắt

Bối cảnh

Tại Việt Nam, sự gia tăng của các bệnh liên quan đến dinh dưỡng như thừa cân, béo phì, đái tháo đường và các vấn đề về tim mạch đang trở thành mối quan tâm sức khỏe cộng đồng đáng kể. Một trong những nguyên nhân chính là thói quen ăn uống thiếu kiểm soát và việc thiếu công cụ hỗ trợ theo dõi dinh dưỡng phù hợp với ẩm thực Việt Nam. Hầu hết các ứng dụng theo dõi dinh dưỡng hiện có trên thị trường đều được thiết kế cho người dùng phương Tây, với cơ sở dữ liệu thực phẩm tập trung vào các món ăn Âu — Mỹ. Điều này gây khó khăn cho người Việt khi muốn ghi nhận các món ăn truyền thống như phở, bún bò, cơm tấm, hay các món xào đặc trưng. Bên cạnh đó, việc nhập liệu thủ công thông tin dinh dưỡng cho từng món ăn là một rào cản lớn về thời gian và công sức, khiến người dùng khó duy trì thói quen theo dõi lâu dài. Từ những thực trạng trên, một ứng dụng di động hỗ trợ theo dõi dinh dưỡng được xây dựng riêng cho người Việt, có khả năng nhận diện món ăn qua hình ảnh bằng trí tuệ nhân tạo, là một nhu cầu thiết thực.

Mục tiêu

Dự án SmartNutri hướng đến mục tiêu xây dựng một ứng dụng di động hỗ trợ người Việt theo dõi dinh dưỡng hằng ngày một cách thuận tiện và trực quan. Cụ thể, ứng dụng cung cấp các chức năng: đăng ký và xác thực người dùng, khảo sát mục tiêu dinh dưỡng cá nhân qua quy trình onboarding, ghi nhật ký ăn uống hằng ngày, theo dõi lượng calo và các chất dinh dưỡng đa lượng (protein, carbohydrate, chất béo), nhận diện món ăn qua ảnh chụp bằng trí tuệ nhân tạo, trò chuyện với chatbot AI để được tư vấn dinh dưỡng, và quản lý danh sách món ăn yêu thích. Ngoài ra, hệ thống còn bao gồm một trang quản trị web dành cho quản trị viên để quản lý cơ sở dữ liệu thực phẩm và người dùng.

Giải pháp

SmartNutri được thiết kế theo kiến trúc ba tầng (three-tier architecture). Tầng trình bày là ứng dụng di động được phát triển bằng Flutter, hỗ trợ nền tảng Android, và trang quản trị web xây dựng bằng React kết hợp TypeScript và Vite. Tầng ứng dụng bao gồm các Cloud Functions chạy trên nền tảng Firebase, đảm nhiệm các tác vụ yêu cầu xử lý phía máy chủ như nhận diện món ăn qua ảnh, gợi ý bữa ăn, tra cứu mã vạch sản phẩm và các thao tác quản trị. Google Gemini API — mô hình ngôn ngữ lớn đa phương thức của Google — được tích hợp để phân tích ảnh món ăn và cung cấp thông tin dinh dưỡng ước tính, cũng như trả lời câu hỏi tư vấn dinh dưỡng qua chatbot. Tầng dữ liệu sử dụng Cloud Firestore làm cơ sở dữ liệu chính với cấu trúc NoSQL dạng document, Firebase Storage để lưu trữ hình ảnh, và Firebase Authentication để xác thực người dùng.

Công nghệ sử dụng

Ứng dụng di động được xây dựng bằng Flutter (Dart), một framework đa nền tảng cho phép biên dịch sang native code với hiệu năng cao. Hệ thống backend sử dụng các dịch vụ của Firebase bao gồm Authentication (xác thực qua email/mật khẩu và Google Sign-In), Cloud Firestore (cơ sở dữ liệu thời gian thực), Firebase Storage (lưu trữ file) và Cloud Functions (môi trường serverless với Node.js và TypeScript). Trang quản trị web được phát triển bằng React 18, TypeScript và Vite. Google Gemini API (model Gemini 2.5 Flash) được sử dụng cho các tác vụ AI: nhận diện món ăn từ ảnh, gợi ý bữa ăn theo mục tiêu dinh dưỡng và trò chuyện tư vấn dinh dưỡng.

Kết quả đạt được

Dự án đã xây dựng thành công một ứng dụng di động hoạt động trên nền tảng Android với giao diện tiếng Việt hoàn chỉnh. Các chức năng chính bao gồm: đăng ký và đăng nhập qua email hoặc tài khoản Google; quy trình onboarding thu thập thông tin cá nhân và thiết lập mục tiêu dinh dưỡng; quản lý hồ sơ người dùng; ghi nhật ký ăn uống theo ngày với khả năng thêm món từ danh mục thực phẩm hoặc nhập thủ công; quét và nhận diện món ăn qua ảnh sử dụng Gemini API; theo dõi lượng calo và chất dinh dưỡng đa lượng dưới dạng biểu đồ; chatbot AI tư vấn dinh dưỡng; quản lý danh sách món ăn yêu thích; và hệ thống phân quyền gói Free/Premium. Trang quản trị web hỗ trợ quản trị viên thực hiện các thao tác CRUD đối với cơ sở dữ liệu thực phẩm, quản lý người dùng và phân quyền. Hệ thống được triển khai trên hạ tầng đám mây của Firebase.

Ý nghĩa

SmartNutri mang lại giá trị thực tiễn cho người dùng Việt Nam khi cung cấp một công cụ theo dõi dinh dưỡng được thiết kế phù hợp với văn hóa ẩm thực trong nước. Khả năng nhận diện món ăn qua ảnh bằng AI giúp giảm đáng kể thời gian và công sức nhập liệu, từ đó khuyến khích người dùng duy trì thói quen theo dõi sức khỏe lâu dài. Về mặt kỹ thuật, dự án minh họa một mô hình phát triển ứng dụng di động hiện đại, kết hợp giữa framework đa nền tảng, dịch vụ đám mây serverless và trí tuệ nhân tạo — một hướng tiếp cận có thể áp dụng cho nhiều lĩnh vực khác ngoài dinh dưỡng.

Mục lục

Lời cảm ơn	2
Lời cam đoan	3
Tóm tắt	4
Danh sách hình vẽ	10
Danh sách bảng	11
Danh mục các từ viết tắt	12
1 Đặt vấn đề	13
1.1 Mô tả bài toán	13
1.2 Mục tiêu dự án	14
1.2.1 Mục tiêu tổng thể	14
1.2.2 Mục tiêu cụ thể	14
1.2.3 Đối tượng người dùng	15
1.3 Cấu trúc báo cáo	15
2 Kiến thức nền tảng	16
2.1 Tổng quan về kiến trúc hệ thống	16
2.1.1 Tầng trình bày (Presentation Tier)	16
2.1.2 Tầng ứng dụng (Application Tier)	16
2.1.3 Tầng dữ liệu (Data Tier)	17
2.1.4 Luồng xử lý quét món ăn bằng AI	18
2.1.5 Cơ chế giao tiếp giữa các tầng	18
2.2 Các công nghệ được sử dụng	19
2.2.1 Firebase	19
2.2.1.1 Firebase Authentication	19
2.2.1.2 Cloud Firestore	19
2.2.1.3 Firebase Storage	20

2.2.1.4	Firebase Cloud Functions	20
2.2.2	Google Gemini API	20
2.2.3	Flutter	21
2.2.4	React, TypeScript và Vite (Admin Web)	21
2.2.5	Các công nghệ hỗ trợ	22
3	Phân tích thiết kế hệ thống	23
3.1	Tổng quan hệ thống	23
3.2	Đặc tả yêu cầu	23
3.2.1	Yêu cầu chức năng	23
3.2.2	Yêu cầu phi chức năng	25
3.2.2.1	Hiệu năng	25
3.2.2.2	Độ tin cậy và tính sẵn sàng	25
3.2.2.3	An ninh và bảo mật	25
3.2.2.4	Khả bảo trì	26
3.2.2.5	Khả mở rộng	26
3.2.2.6	Khả sử dụng	26
3.2.2.7	Ràng buộc kỹ thuật	27
3.3	Tổng quan ca sử dụng hệ thống	27
3.3.1	Biểu đồ ca sử dụng cho Người dùng	27
3.3.2	Biểu đồ ca sử dụng cho Quản trị viên	28
3.4	Mô tả ca sử dụng	29
3.4.1	UC-01: Đăng ký tài khoản	29
3.4.2	UC-02: Đăng nhập	30
3.4.3	UC-05: Quét món ăn bằng AI	31
3.4.4	UC-06: Nhật ký ăn uống	34
3.4.5	UC-07: Theo dõi dinh dưỡng	36
3.4.6	UC-03: Quản lý hồ sơ cá nhân	38
3.4.7	UC-04: Hoàn thành Onboarding	39
3.4.8	UC-08: Quản lý món ăn yêu thích	40
3.4.9	UC-09: Chatbot AI dinh dưỡng	41
3.4.10	UC-10: Quản lý thực phẩm	44
3.4.11	UC-11: Quản lý người dùng	45
3.4.12	UC-12: Quản lý trạng thái Premium	46
3.5	Thiết kế cơ sở dữ liệu	48
3.5.1	Về sơ đồ ERD	48
3.5.2	Nguyên tắc phân tách dữ liệu người dùng	49
3.5.3	Mô tả các collection chính	49

3.5.3.1	Collection: users	49
3.5.3.2	Collection: foods	50
3.5.3.3	Sub-collection: users/{userId}/meal_entries	51
3.5.3.4	Sub-collection: users/{userId}/favorites	51
3.5.3.5	Sub-collection: users/{userId}/usage	52
3.5.3.6	Sub-collection: users/{userId}/chat_history	52
4	Xây dựng, triển khai và kiểm thử hệ thống	53
4.1	Cài đặt hệ thống	53
4.1.1	Môi trường phát triển	53
4.1.2	Cài đặt ứng dụng di động (Flutter)	53
4.1.3	Cài đặt Firebase backend	55
4.1.4	Cài đặt Cloud Functions	56
4.1.5	Cài đặt trang Admin Web	57
4.1.6	Cấu hình Gemini API	58
4.2	Triển khai hệ thống	58
4.2.1	Trạng thái triển khai hiện tại	59
4.2.2	Quy trình triển khai	60
4.3	Kiểm thử hệ thống và trình bày kết quả thực nghiệm	60
4.3.1	Kiểm thử chức năng	60
4.3.2	Kiểm thử chức năng quét món ăn bằng AI	66
4.3.2.1	Nhận xét về kết quả kiểm thử quét AI	68
4.3.3	Kiểm thử hiệu năng	69
4.3.3.1	Nhận xét về kiểm thử hiệu năng	71
4.3.3.2	Nhận xét về kiểm thử bảo mật	74
4.3.4	Kiểm thử tích hợp	75
4.3.5	Kiểm thử giao diện	75
4.4	Giao diện hệ thống	75
4.4.1	Các màn hình chính của ứng dụng	75
	Kết luận	79
	Tài liệu tham khảo	82

Danh sách hình vẽ

2.1	Kiến trúc tổng thể hệ thống SmartNutri	17
3.1	Biểu đồ ca sử dụng cho Người dùng — Hệ thống SmartNutri	28
3.2	Biểu đồ ca sử dụng cho Quản trị viên — Hệ thống SmartNutri	29
3.3	Biểu đồ tuần tự ca sử dụng Quét món ăn bằng AI	34
3.4	Biểu đồ tuần tự ca sử dụng Nhật ký ăn uống	36
3.5	Biểu đồ tuần tự ca sử dụng Theo dõi dinh dưỡng	38
3.6	Biểu đồ tuần tự ca sử dụng Chatbot AI dinh dưỡng	44
3.7	Sơ đồ quan hệ logic giữa các nhóm dữ liệu trong SmartNutri	49
4.1	Kiến trúc phân tầng ứng dụng Flutter	55
4.2	Màn hình đăng nhập	75
4.3	Màn hình tổng quan (trang chủ)	76
4.4	Màn hình quản lý nhật ký bữa ăn	77
4.5	Màn hình trợ lý dinh dưỡng AI	78

Danh sách bảng

4.1	Các thành phần triển khai của hệ thống SmartNutri	58
4.2	Kết quả kiểm thử chức năng hệ thống SmartNutri	61
4.3	Kiểm thử nhận diện món ăn Việt Nam bằng Gemini API qua Cloud Function .	67
4.4	Khung kiểm thử hiệu năng hệ thống SmartNutri	70
4.5	Kiểm thử bảo mật hệ thống SmartNutri	72

Danh mục các từ viết tắt

STT	Từ viết tắt	Từ đầy đủ tiếng Anh	Nghĩa tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	API	Application Programming Interface	Giao diện lập trình ứng dụng
3	CRUD	Create, Read, Update, Delete	Tạo, Đọc, Cập nhật, Xóa
4	CSS	Cascading Style Sheets	Ngôn ngữ định kiểu
5	DB	Database	Cơ sở dữ liệu
6	ERD	Entity-Relationship Diagram	Biểu đồ thực thể - liên kết
7	JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu
8	JWT	JSON Web Token	Token xác thực JSON
9	MVC	Model-View-Controller	Mô hình - Giao diện - Điều khiển
10	NoSQL	Not Only SQL	Cơ sở dữ liệu phi quan hệ
11	REST	Representational State Transfer	Kiến trúc truyền tải trạng thái
12	SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
13	SSL/TLS	Secure Sockets Layer / Transport Layer Security	Giao thức bảo mật tầng vận chuyển
14	UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
15	URL	Uniform Resource Locator	Địa chỉ tài nguyên

CHƯƠNG 1

Đặt vấn đề

1.1 Mô tả bài toán

Những năm gần đây, nhận thức của người Việt về vai trò của dinh dưỡng đối với sức khỏe đã được nâng cao đáng kể. Các vấn đề như thừa cân, béo phì, rối loạn chuyển hóa và các bệnh mãn tính liên quan đến chế độ ăn uống đang có xu hướng gia tăng tại Việt Nam. Điều này thúc đẩy nhu cầu về các công cụ hỗ trợ theo dõi và quản lý dinh dưỡng cá nhân một cách thuận tiện và chính xác.

Tuy nhiên, việc theo dõi dinh dưỡng hằng ngày tại Việt Nam hiện đang gặp phải một số trở ngại đáng kể:

- **Khó ước lượng dinh dưỡng trong món ăn Việt:** Ẩm thực Việt Nam rất đa dạng, với hàng trăm món ăn khác nhau giữa các vùng miền. Mỗi món thường là sự kết hợp của nhiều nguyên liệu, gia vị và cách chế biến khác nhau, khiến người dùng rất khó ước lượng chính xác lượng calo và các chất dinh dưỡng đa lượng (protein, carbohydrate, chất béo) có trong khẩu phần mình tiêu thụ. Ngay cả với những người có kiến thức dinh dưỡng cơ bản, việc ước lượng thành phần của một tô phở, một đĩa cơm tấm hay một phần bún thịt nướng vẫn là một thách thức.
- **Ghi chép thủ công mất thời gian:** Phương pháp ghi chép nhật ký ăn uống truyền thống — dù là trên giấy hay trên các ứng dụng — đều yêu cầu người dùng phải nhập liệu thủ công từng món ăn, tra cứu thông tin dinh dưỡng và ước lượng khẩu phần. Quá trình này tốn nhiều thời gian và công sức, dễ dẫn đến tình trạng người dùng bỏ cuộc sau một thời gian ngắn sử dụng.
- **Ứng dụng nước ngoài chưa phù hợp:** Các ứng dụng theo dõi dinh dưỡng phổ biến trên thị trường như MyFitnessPal, Lose It! hay Cronometer được phát triển chủ yếu cho thị trường phương Tây. Cơ sở dữ liệu thực phẩm của các ứng dụng này tập trung vào các

món ăn Âu — Mỹ, thiếu vắng hoặc có rất ít thông tin về các món ăn Việt Nam. Bên cạnh đó, giao diện và ngôn ngữ hiển thị bằng tiếng Anh cũng là một rào cản đối với phần lớn người dùng Việt.

- **Thiếu giải pháp tích hợp AI cho món Việt:** Hiện chưa có một ứng dụng nào kết hợp được ba yếu tố: hỗ trợ tiếng Việt hoàn toàn, khả năng nhận diện món ăn Việt qua hình ảnh bằng trí tuệ nhân tạo, và cơ sở dữ liệu dinh dưỡng dành riêng cho ẩm thực Việt Nam. Khoảng trống này tạo ra nhu cầu về một giải pháp được thiết kế riêng cho người Việt, tận dụng sức mạnh của AI để đơn giản hóa việc theo dõi dinh dưỡng hằng ngày.

Từ những thực trạng trên, việc xây dựng ứng dụng SmartNutri — một ứng dụng di động hỗ trợ tiếng Việt, tích hợp trí tuệ nhân tạo để nhận diện món ăn qua ảnh chụp và theo dõi dinh dưỡng — là hướng đi thiết thực, đáp ứng đúng nhu cầu của người dùng Việt Nam trong bối cảnh hiện nay.

1.2 Mục tiêu dự án

1.2.1 Mục tiêu tổng thể

Mục tiêu tổng thể của dự án là xây dựng ứng dụng di động SmartNutri — một nền tảng theo dõi dinh dưỡng thông minh dành cho người Việt, tích hợp trí tuệ nhân tạo để nhận diện món ăn qua hình ảnh và hỗ trợ người dùng theo dõi, quản lý chế độ ăn uống hằng ngày một cách thuận tiện.

1.2.2 Mục tiêu cụ thể

1. **Xác thực người dùng:** Xây dựng hệ thống đăng ký và đăng nhập qua email/mật khẩu và tài khoản Google, sử dụng Firebase Authentication.
2. **Quản lý hồ sơ và mục tiêu dinh dưỡng:** Cho phép người dùng nhập thông tin cá nhân (tuổi, giới tính, chiều cao, cân nặng, mức độ vận động) và thiết lập mục tiêu dinh dưỡng (calo, protein, carbohydrate, chất béo) thông qua quy trình onboarding.
3. **Ghi nhật ký ăn uống:** Cung cấp công cụ để người dùng ghi lại các món ăn đã tiêu thụ trong ngày, bao gồm thêm món từ danh mục thực phẩm có sẵn và nhập thủ công.
4. **Quét món ăn bằng AI:** Tích hợp Google Gemini API để nhận diện món ăn từ ảnh chụp, trả về thông tin dinh dưỡng ước tính và tự động thêm vào nhật ký.
5. **Theo dõi dinh dưỡng:** Hiển thị lượng calo, protein, carbohydrate và chất béo đã tiêu thụ theo ngày dưới dạng biểu đồ, so sánh với mục tiêu đã đặt ra.

6. **Chatbot AI tư vấn dinh dưỡng:** Xây dựng chatbot tích hợp Gemini API để trả lời câu hỏi về dinh dưỡng ở mức tham khảo, dựa trên hồ sơ và mục tiêu của người dùng.
7. **Trang quản trị:** Xây dựng giao diện web cho phép quản trị viên quản lý cơ sở dữ liệu thực phẩm (thêm, sửa, xóa, import) và quản lý người dùng (xem danh sách, phân quyền, gán gói Premium).

Bên cạnh các mục tiêu đã triển khai, dự án cũng đặt nền tảng ban đầu cho một số định hướng phát triển trong tương lai như gợi ý bữa ăn theo mục tiêu dinh dưỡng và hệ thống gói Premium với các tính năng nâng cao.

1.2.3 Đối tượng người dùng

- Người dùng phổ thông quan tâm đến sức khỏe và có nhu cầu theo dõi dinh dưỡng hằng ngày.
- Người đang trong quá trình giảm cân, tăng cân hoặc duy trì cân nặng.
- Người tập gym, vận động viên cần theo dõi lượng chất dinh dưỡng đa lượng.
- Quản trị viên hệ thống thực hiện quản lý dữ liệu thực phẩm và người dùng qua trang web.

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức thành bốn chương chính:

- **Chương 1 — Đặt vấn đề:** Giới thiệu bối cảnh, mục tiêu và phạm vi của dự án.
- **Chương 2 — Kiến thức nền tảng:** Trình bày các kiến thức và công nghệ nền tảng được sử dụng trong dự án.
- **Chương 3 — Phân tích thiết kế hệ thống:** Phân tích yêu cầu, đặc tả use case và thiết kế cơ sở dữ liệu.
- **Chương 4 — Xây dựng, triển khai và kiểm thử:** Mô tả quá trình cài đặt, triển khai và kết quả kiểm thử hệ thống.

CHƯƠNG 2

Kiến trúc nền tảng

2.1 Tổng quan về kiến trúc hệ thống

Hệ thống SmartNutri được thiết kế theo mô hình **ba tầng** (three-tier architecture), tách biệt rõ trách nhiệm giữa giao diện, logic xử lý và lưu trữ dữ liệu.

2.1.1 Tầng trình bày (Presentation Tier)

Tầng trình bày gồm hai thành phần chính:

- **Ứng dụng di động Flutter:** Là giao diện chính cho người dùng cuối, cung cấp các màn hình đăng ký/đăng nhập, onboarding, trang chủ, nhật ký ăn uống, quét món ăn, thống kê dinh dưỡng, chatbot AI và quản lý hồ sơ. Ứng dụng giao tiếp trực tiếp với Firebase Authentication và Firestore thông qua Firebase SDK, sử dụng Provider/ChangeNotifier để quản lý trạng thái và cập nhật UI khi dữ liệu thay đổi.
- **Trang quản trị web (React + TypeScript + Vite):** Dành cho quản trị viên, cung cấp giao diện quản lý cơ sở dữ liệu thực phẩm (thêm, sửa, xóa, import CSV) và quản lý người dùng (phân quyền admin, gán gói Premium). Trang admin giao tiếp với Firestore và Cloud Functions thông qua Firebase Web SDK.

2.1.2 Tầng ứng dụng (Application Tier)

Tầng ứng dụng bao gồm Firebase Cloud Functions và Google Gemini API, đảm nhận các tác vụ yêu cầu xử lý phía máy chủ:

- **Firebase Cloud Functions:** Đóng vai trò là tầng logic trung gian giữa client và các dịch vụ bên ngoài. Các nghiệp vụ backend như nhận diện món ăn bằng AI, gợi ý bữa ăn, phân

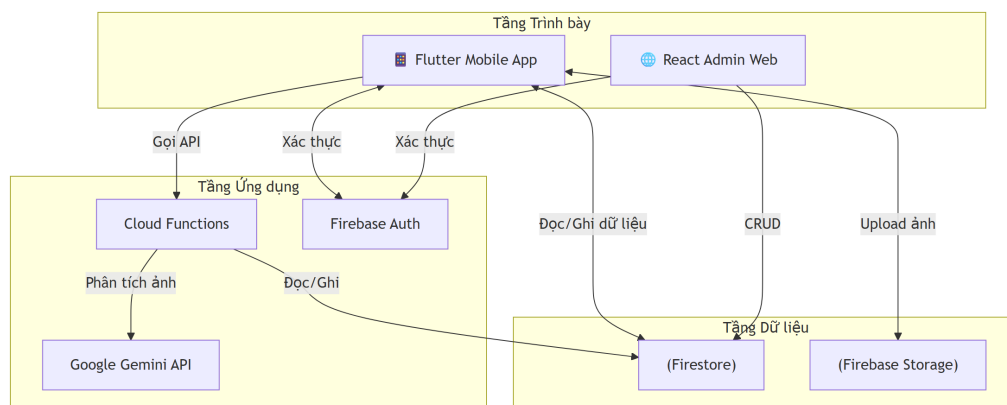
quyền người dùng và import dữ liệu được đóng gói trong các Cloud Function riêng biệt, gọi được từ cả ứng dụng di động lẫn trang admin thông qua HTTP request có kèm token xác thực.

- **Google Gemini API:** Được gọi từ phía Cloud Functions (không gọi trực tiếp từ client) để thực hiện nhận diện món ăn từ ảnh và trả lời câu hỏi tư vấn dinh dưỡng. Việc gọi Gemini qua Cloud Functions giúp bảo vệ API key — khóa được lưu dưới dạng biến môi trường trên Cloud Functions và không bao giờ xuất hiện trong mã nguồn phía client.
- **Firebase Authentication:** Xử lý xác thực người dùng (email/password và Google Sign-In), quản lý phiên đăng nhập và cấp token để xác định danh tính người dùng trong các request đến Firestore và Cloud Functions.

2.1.3 Tầng dữ liệu (Data Tier)

Tầng dữ liệu được xây dựng trên hai dịch vụ của Firebase:

- **Cloud Firestore:** Cơ sở dữ liệu NoSQL dạng document, lưu trữ toàn bộ dữ liệu của hệ thống. Các collection chính gồm: users (thông tin tài khoản, subscription, aiScanUsage), foods (danh mục thực phẩm), và các subcollection như users/{uid}/meal_entries (nhật ký ăn uống), users/{uid}/favorites (món yêu thích), users/{uid}/usage (quota quét AI theo tháng), users/{uid}/chat_history (lịch sử trò chuyện với AI). Dữ liệu được phân tách theo UID, đảm bảo mỗi người dùng chỉ truy cập được dữ liệu của chính mình thông qua Firestore Security Rules.
- **Firebase Storage:** Lưu trữ file tĩnh, chủ yếu là ảnh đại diện người dùng và ảnh món ăn tải lên trong quá trình quét AI, được tổ chức theo đường dẫn users/{uid}/.



Hình 2.1: Kiến trúc tổng thể hệ thống SmartNutri

2.1.4 Luồng xử lý quét món ăn bằng AI

Để minh họa cách các tầng phối hợp với nhau, dưới đây là luồng xử lý của tính năng quét món ăn — một trong những chức năng cốt lõi của hệ thống:

1. Người dùng chụp hoặc chọn ảnh món ăn từ ứng dụng Flutter (tầng trình bày).
2. Ảnh được resize và chuyển thành chuỗi base64, sau đó gửi đến Cloud Function `identifyFoodImage` qua HTTP request kèm Firebase Auth token.
3. Cloud Function (tầng ứng dụng) xác thực token, kiểm tra quota quét AI của người dùng trong Firestore (tầng dữ liệu). Nếu người dùng gói free đã hết lượt quét trong tháng, Cloud Function từ chối yêu cầu và trả về lỗi.
4. Nếu còn quota, Cloud Function gửi ảnh kèm prompt phân tích đến Gemini API. Prompt được thiết kế để Gemini trả về danh sách món ăn dưới dạng JSON có cấu trúc, bao gồm tên món, calo ước tính, protein, carbohydrate, chất béo, khối lượng phần ăn và độ tự tin.
5. Gemini API trả về kết quả cho Cloud Function. Cloud Function cập nhật số lượt quét đã sử dụng trong Firestore, sau đó trả kết quả về ứng dụng.
6. Ứng dụng hiển thị kết quả cho người dùng. Người dùng có thể xác nhận để thêm món ăn vào nhật ký (lưu vào subcollection `meal_entries` trong Firestore) hoặc chỉnh sửa thông tin dinh dưỡng trước khi lưu.

2.1.5 Cơ chế giao tiếp giữa các tầng

Hệ thống sử dụng hai cơ chế giao tiếp chính:

- **Firebase SDK (trực tiếp):** Ứng dụng Flutter và trang admin giao tiếp trực tiếp với Firestore, Storage và Authentication thông qua Firebase SDK. Đây là cơ chế chính cho hầu hết các thao tác đọc/ghi dữ liệu người dùng.
- **Cloud Functions (HTTP request):** Các tác vụ cần xử lý phía máy chủ hoặc gọi dịch vụ bên ngoài (như Gemini API) được đóng gói trong Cloud Function và gọi qua HTTP request có kèm token xác thực Firebase.

Trong ứng dụng Flutter, code được tổ chức theo cấu trúc **feature-first**: mỗi tính năng (đăng nhập, nhật ký ăn uống, quét món ăn, chatbot,...) được đặt trong một thư mục riêng, gồm hai tầng con là *domain* (model, service) và *presentation* (widget, page).

2.2 Các công nghệ được sử dụng

2.2.1 Firebase

Firebase là nền tảng phát triển ứng dụng của Google, cung cấp các dịch vụ backend dưới dạng BaaS (Backend-as-a-Service). SmartNutri lựa chọn Firebase làm nền tảng backend chính vì khả năng tích hợp sẵn giữa các dịch vụ và mô hình lập trình phía client, giúp nhóm phát triển tập trung vào logic nghiệp vụ thay vì quản lý hạ tầng máy chủ.

2.2.1.1 Firebase Authentication

Firebase Authentication [1] cung cấp hệ thống xác thực người dùng với nhiều phương thức đăng nhập. Trong SmartNutri, dịch vụ này được sử dụng để:

- Cho phép người dùng đăng ký và đăng nhập bằng email/mật khẩu.
- Hỗ trợ đăng nhập nhanh qua tài khoản Google.
- Tự động quản lý phiên đăng nhập và refresh token, đảm bảo người dùng không phải đăng nhập lại mỗi lần mở ứng dụng.

Mỗi người dùng sau khi xác thực được gán một mã định danh duy nhất (UID), mã này được dùng làm khóa chính cho toàn bộ dữ liệu cá nhân trong Firestore, đảm bảo dữ liệu được phân tách rõ ràng giữa những người dùng khác nhau.

2.2.1.2 Cloud Firestore

Cloud Firestore [2] là cơ sở dữ liệu NoSQL dạng document của Firebase. Dữ liệu được tổ chức thành các collection, mỗi collection chứa nhiều document — mỗi document là một bản ghi JSON linh hoạt, không yêu cầu schema cố định. Firestore được chọn cho SmartNutri vì các lý do sau:

- **Cấu trúc dữ liệu theo từng người dùng:** Mô hình subcollection của Firestore (ví dụ: `users/{uid}/meal_entries/{entryId}`) cho phép tổ chức dữ liệu nhật ký ăn uống, món yêu thích, lịch sử chat của từng người dùng một cách tự nhiên và hiệu quả.
- **Đồng bộ thời gian thực:** Khi người dùng thêm một món ăn vào nhật ký, dữ liệu được cập nhật ngay lập tức trên giao diện mà không cần thao tác làm mới thủ công, nhờ cơ chế snapshot listener của Firestore.

- **Security Rules:** Cho phép định nghĩa quy tắc bảo mật trực tiếp trên Firestore, đảm bảo mỗi người dùng chỉ có thể đọc và ghi dữ liệu thuộc về chính mình.
- **Hỗ trợ offline:** Firestore tự động lưu đệm dữ liệu trên thiết bị, cho phép ứng dụng hoạt động ngay cả khi không có kết nối mạng và tự động đồng bộ khi kết nối được khôi phục.

2.2.1.3 Firebase Storage

Firebase Storage [3] cung cấp dịch vụ lưu trữ file trên đám mây, tích hợp với Firebase Authentication để kiểm soát quyền truy cập. Trong SmartNutri, Storage được sử dụng để lưu trữ ảnh đại diện (avatar) của người dùng và ảnh món ăn do người dùng tải lên trong quá trình sử dụng tính năng quét AI. Các file được tổ chức theo cấu trúc thư mục `users/{uid}/` để đảm bảo mỗi người dùng chỉ truy cập được file của chính mình.

2.2.1.4 Firebase Cloud Functions

Cloud Functions [4] là môi trường serverless cho phép chạy code backend mà không cần quản lý máy chủ. Trong SmartNutri, Cloud Functions đảm nhận các tác vụ không nên thực hiện trực tiếp từ phía client:

- **Nhận diện món ăn bằng AI:** Hàm `identifyFoodImage` nhận ảnh từ client, gọi Gemini API để phân tích và trả về kết quả. Việc gọi Gemini qua Cloud Functions thay vì gọi trực tiếp từ ứng dụng giúp bảo vệ API key khỏi bị lộ ra phía client.
- **Gợi ý bữa ăn:** Hàm `suggestMeals` nhận thông tin dinh dưỡng còn thiếu của người dùng, truy vấn cơ sở dữ liệu thực phẩm và sử dụng Gemini để đề xuất các món ăn phù hợp.
- **Thao tác quản trị:** Các hàm `setUserSubscription` và `setAdminRole` cho phép quản trị viên phân quyền người dùng, được bảo vệ bởi cơ chế kiểm tra quyền admin.
- **Import dữ liệu thực phẩm:** Hàm `seedFoods` cho phép nhập hàng loạt thực phẩm vào Firestore theo định dạng JSON, hỗ trợ quá trình xây dựng cơ sở dữ liệu món ăn.

2.2.2 Google Gemini API

Gemini là mô hình ngôn ngữ lớn (LLM) đa phương thức do Google phát triển, có khả năng xử lý đồng thời văn bản và hình ảnh [5]. Trong SmartNutri, Gemini API (model `gemini-2.5-flash`) được sử dụng cho hai tác vụ chính:

- **Nhận diện món ăn từ ảnh:** Khi người dùng chụp ảnh món ăn, ảnh được gửi kèm prompt mô tả yêu cầu phân tích chi tiết (màu sắc, hình dạng, thành phần, cách trình bày). Gemini trả về tên món ăn, lượng calo ước tính, các chất dinh dưỡng đa lượng và khối lượng phần ăn dưới dạng JSON có cấu trúc. Prompt được thiết kế để ưu tiên khớp tên món với danh sách thực phẩm đã có trong cơ sở dữ liệu của SmartNutri, đảm bảo tính nhất quán của dữ liệu.
- **Chatbot tư vấn dinh dưỡng:** Người dùng có thể đặt câu hỏi về dinh dưỡng và nhận câu trả lời từ Gemini. Trước khi gửi câu hỏi, hệ thống đính kèm ngữ cảnh gồm hồ sơ cá nhân, mục tiêu dinh dưỡng và nhật ký ăn uống gần đây của người dùng, giúp câu trả lời mang tính cá nhân hóa và phù hợp với tình trạng thực tế.

2.2.3 Flutter

Flutter [6] là framework phát triển ứng dụng đa nền tảng của Google, sử dụng ngôn ngữ Dart [7]. SmartNutri chọn Flutter vì:

- **Đa nền tảng từ một codebase:** Cùng một mã nguồn Dart có thể biên dịch sang cả Android và iOS. Hiện tại SmartNutri đã hoạt động trên Android; việc hỗ trợ iOS trong tương lai không yêu cầu viết lại ứng dụng.
- **Tích hợp Firebase thuận lợi:** Flutter có bộ plugin chính thức cho Firebase (`firebase_auth`, `cloud_firestore`, `firebase_storage`), giúp việc kết nối và thao tác với các dịch vụ Firebase trở nên đơn giản và ổn định.
- **Hiệu năng cao:** Flutter biên dịch trực tiếp sang native code ARM, không sử dụng bridge JavaScript, phù hợp với các thao tác yêu cầu phản hồi nhanh như hiển thị danh sách món ăn hay biểu đồ dinh dưỡng.
- **Hot Reload:** Cho phép xem kết quả thay đổi code ngay lập tức trên thiết bị hoặc trình giả lập, rút ngắn đáng kể thời gian phát triển và gỡ lỗi giao diện.

2.2.4 React, TypeScript và Vite (Admin Web)

Trang quản trị của SmartNutri được xây dựng bằng React 18 [8], TypeScript [9] và Vite [10]. Trang này phục vụ hai mục đích chính:

- **Quản lý cơ sở dữ liệu thực phẩm:** Quản trị viên có thể thêm, sửa, xóa và import hàng loạt thực phẩm qua giao diện web. Dữ liệu được ghi trực tiếp vào collection `foods` trong Firestore và được ứng dụng di động truy vấn để hiển thị cho người dùng.

- **Quản lý người dùng:** Quản trị viên có thể xem danh sách người dùng đã đăng ký, gán gói Premium cho người dùng và phân quyền admin thông qua các Cloud Functions.

React được chọn vì khả năng xây dựng giao diện dạng SPA (Single Page Application) với hiệu năng tốt. TypeScript cung cấp kiểm tra kiểu tĩnh, giảm thiểu lỗi khi thao tác với dữ liệu Firestore vốn có cấu trúc linh hoạt. Vite làm công cụ build với thời gian khởi động nhanh và hỗ trợ HMR (Hot Module Replacement), giúp quá trình phát triển trang admin diễn ra thuận lợi.

2.2.5 Các công nghệ bổ trợ

- **Node.js:** Môi trường runtime cho Firebase Cloud Functions, sử dụng phiên bản 20 LTS.
- **Git & GitHub:** Quản lý phiên bản và lưu trữ mã nguồn của toàn bộ dự án.
- **Firebase Emulator Suite:** Bộ công cụ giả lập các dịch vụ Firebase trên máy local (Auth, Firestore, Storage, Functions), phục vụ phát triển và kiểm thử mà không ảnh hưởng đến dữ liệu production.
- **Android Emulator:** Môi trường kiểm thử ứng dụng trên máy ảo Android trong quá trình phát triển.

CHƯƠNG 3

Phân tích thiết kế hệ thống

3.1 Tổng quan hệ thống

Hệ thống SmartNutri bao gồm ba thành phần chính:

1. **Ứng dụng di động (Mobile App):** Ứng dụng Flutter chạy trên Android/iOS, là điểm tương tác chính với người dùng cuối.
2. **Backend Firebase:** Bao gồm Authentication, Firestore, Storage và Cloud Functions, xử lý toàn bộ logic nghiệp vụ và lưu trữ dữ liệu.
3. **Trang quản trị (Admin Web):** Ứng dụng React cho phép quản trị viên quản lý dữ liệu thực phẩm, người dùng và theo dõi hoạt động hệ thống.

Các actor chính của hệ thống:

- **Người dùng (User):** Người dùng cuối sử dụng ứng dụng di động để theo dõi dinh dưỡng.
- **Quản trị viên (Admin):** Người quản lý dữ liệu thực phẩm, danh mục và người dùng qua trang admin web.
- **Hệ thống AI (Gemini):** Dịch vụ AI bên ngoài hỗ trợ nhận diện món ăn và phân tích dinh dưỡng.

3.2 Đặc tả yêu cầu

3.2.1 Yêu cầu chức năng

Use Case	Mô tả
UC-01: Đăng ký tài khoản	Người dùng tạo tài khoản mới bằng email và mật khẩu. Hệ thống kiểm tra tính hợp lệ, gửi email xác minh và tạo document người dùng trong Firestore.
UC-02: Đăng nhập	Người dùng đăng nhập bằng email/mật khẩu hoặc tài khoản Google. Hệ thống xác thực qua Firebase Authentication, tải hồ sơ và chuyển đến màn hình phù hợp (onboarding hoặc trang chủ).
UC-03: Quản lý hồ sơ cá nhân	Người dùng xem và cập nhật thông tin cá nhân (tên, tuổi, giới tính, chiều cao, cân nặng, mức độ vận động). Dữ liệu được lưu trong collection profiles/{uid}.
UC-04: Hoàn thành Onboarding	Người dùng nhập thông tin ban đầu (thể trạng, mức vận động, mục tiêu dinh dưỡng). Hệ thống tính toán chỉ tiêu calo và macro, lưu vào goals/{uid} và đánh dấu hoàn tất onboarding.
UC-05: Quét món ăn bằng AI	Người dùng chụp hoặc chọn ảnh món ăn. Ảnh được gửi đến Cloud Function, Cloud Function gọi Gemini API để nhận diện và trả về tên món, calo và macro ước tính. Người dùng xác nhận trước khi lưu vào nhật ký.
UC-06: Nhật ký ăn uống	Người dùng xem danh sách món ăn đã tiêu thụ trong ngày, thêm món mới từ danh mục thực phẩm hoặc nhập thủ công, và xóa món khỏi nhật ký. Dữ liệu lưu trong users/{uid}/meal_entries.
UC-07: Theo dõi dinh dưỡng	Người dùng xem biểu đồ và số liệu thống kê về lượng calo, protein, carbohydrate, chất béo đã tiêu thụ theo ngày, so sánh với mục tiêu đã đặt.
UC-08: Quản lý món ăn yêu thích	Người dùng thêm món ăn vào danh sách yêu thích để truy cập nhanh khi ghi nhật ký, hoặc bỏ yêu thích. Dữ liệu lưu trong users/{uid}/favorites.
UC-09: Chatbot AI dinh dưỡng	Người dùng đặt câu hỏi về dinh dưỡng và nhận phản hồi từ Gemini API. Hệ thống đính kèm hồ sơ và mục tiêu dinh dưỡng của người dùng làm ngữ cảnh để cá nhân hóa câu trả lời. Lịch sử chat được lưu trong users/{uid}/chat_history.
UC-10: Quản lý thực phẩm (Admin)	Quản trị viên thêm, sửa, xóa thực phẩm trong danh mục foods, và import thực phẩm hàng loạt qua CSV. Dữ liệu được ứng dụng di động truy vấn để hiển thị cho người dùng.
UC-11: Quản lý người dùng (Admin)	Quản trị viên xem danh sách người dùng đã đăng ký, phân quyền admin và gán trạng thái Premium cho người dùng thông qua Cloud Functions.

Use Case	Mô tả
UC-12: Quản lý trạng thái Premium	Hệ thống hỗ trợ quản lý trạng thái gói Premium (Free/Premium) cho mỗi người dùng, bao gồm thời hạn và nguồn gán (admin, mô phỏng). Hiện tại, việc gán Premium được thực hiện qua Cloud Function bởi quản trị viên; tích hợp thanh toán thật (In-app Purchase) là định hướng phát triển tiếp theo.

3.2.2 Yêu cầu phi chức năng

3.2.2.1 Hiệu năng

- Ứng dụng khởi động và hiển thị màn hình chính trong thời gian chấp nhận được (mục tiêu dưới 3 giây trên thiết bị Android tầm trung).
- Truy vấn nhật ký ăn uống theo ngày hiển thị kết quả nhanh, tận dụng Firestore local cache cho dữ liệu đã từng truy cập.
- Thời gian quét món ăn bằng AI phụ thuộc vào độ trễ mạng và thời gian phản hồi của Gemini API; mục tiêu hiển thị kết quả trong vòng 10 giây kể từ khi gửi ảnh.
- Biểu đồ dinh dưỡng được tính toán và hiển thị mượt mà trên tập dữ liệu nhật ký theo tuần.

3.2.2.2 Độ tin cậy và tính sẵn sàng

- Hệ thống backend phụ thuộc vào hạ tầng Firebase của Google; tính sẵn sàng của ứng dụng ở mức tương đương Firebase SLA.
- Dữ liệu người dùng được lưu trên Firestore với cơ chế sao lưu tự động của Firebase.
- Ứng dụng hỗ trợ chế độ offline cơ bản: dữ liệu đã tải được giữ trong Firestore local cache, cho phép người dùng xem nhật ký và danh sách thực phẩm khi không có kết nối mạng. Các thao tác ghi (thêm món, quét AI) yêu cầu kết nối mạng để đồng bộ.

3.2.2.3 An ninh và bảo mật

- Xác thực người dùng qua Firebase Authentication, sử dụng token JWT để xác định danh tính trong mọi request đến Firestore và Cloud Functions.
- Firestore Security Rules đảm bảo mỗi người dùng chỉ có thể đọc và ghi dữ liệu thuộc về chính mình (phân tách theo UID).

- Toàn bộ kết nối giữa client và Firebase sử dụng SSL/TLS.
- Gemini API key được lưu dưới dạng biến môi trường trên Cloud Functions, không xuất hiện trong mã nguồn phía client, tránh nguy cơ bị trích xuất từ file APK.
- Trang admin yêu cầu tài khoản có admin claim mới có thể gọi các Cloud Function quản trị (setAdminRole, setUserSubscription).

3.2.2.4 Khả bảo trì

- Code Flutter được tổ chức theo cấu trúc feature-first: mỗi tính năng nằm trong thư mục riêng, tách biệt giữa domain (model, service) và presentation (widget, page).
- Cloud Functions và Admin Web sử dụng TypeScript, cung cấp kiểm tra kiểu tĩnh, giảm thiểu lỗi runtime và hỗ trợ quá trình bảo trì.
- Các service class trong Flutter đảm nhận toàn bộ logic giao tiếp với Firebase, tránh việc gọi Firebase trực tiếp từ widget.

3.2.2.5 Khả mở rộng

- Cấu trúc feature-first cho phép thêm tính năng mới bằng cách tạo module mới mà không ảnh hưởng đến các module hiện có.
- Firestore là NoSQL document database, không yêu cầu schema cố định, cho phép thêm trường mới vào document mà không cần migrate dữ liệu cũ.
- Cloud Functions có thể được thêm mới hoặc cập nhật độc lập mà không cần phát hành lại ứng dụng di động.
- Kiến trúc hiện tại đã có nền tảng để mở rộng sang iOS khi cần (dùng chung codebase Flutter).

3.2.2.6 Khả sử dụng

- Giao diện người dùng hoàn toàn bằng tiếng Việt, phù hợp với đối tượng người dùng phổ thông tại Việt Nam.
- Luồng thao tác được thiết kế đơn giản: người dùng mới được dẫn qua onboarding từng bước; người dùng đã đăng nhập có thể thêm món ăn vào nhật ký chỉ với vài thao tác.
- Tính năng quét món ăn bằng ảnh giúp giảm thiểu thao tác nhập liệu thủ công.

- Giao diện sử dụng các thành phần Material Design quen thuộc, bố cục nhất quán giữa các màn hình.

3.2.2.7 Ràng buộc kỹ thuật

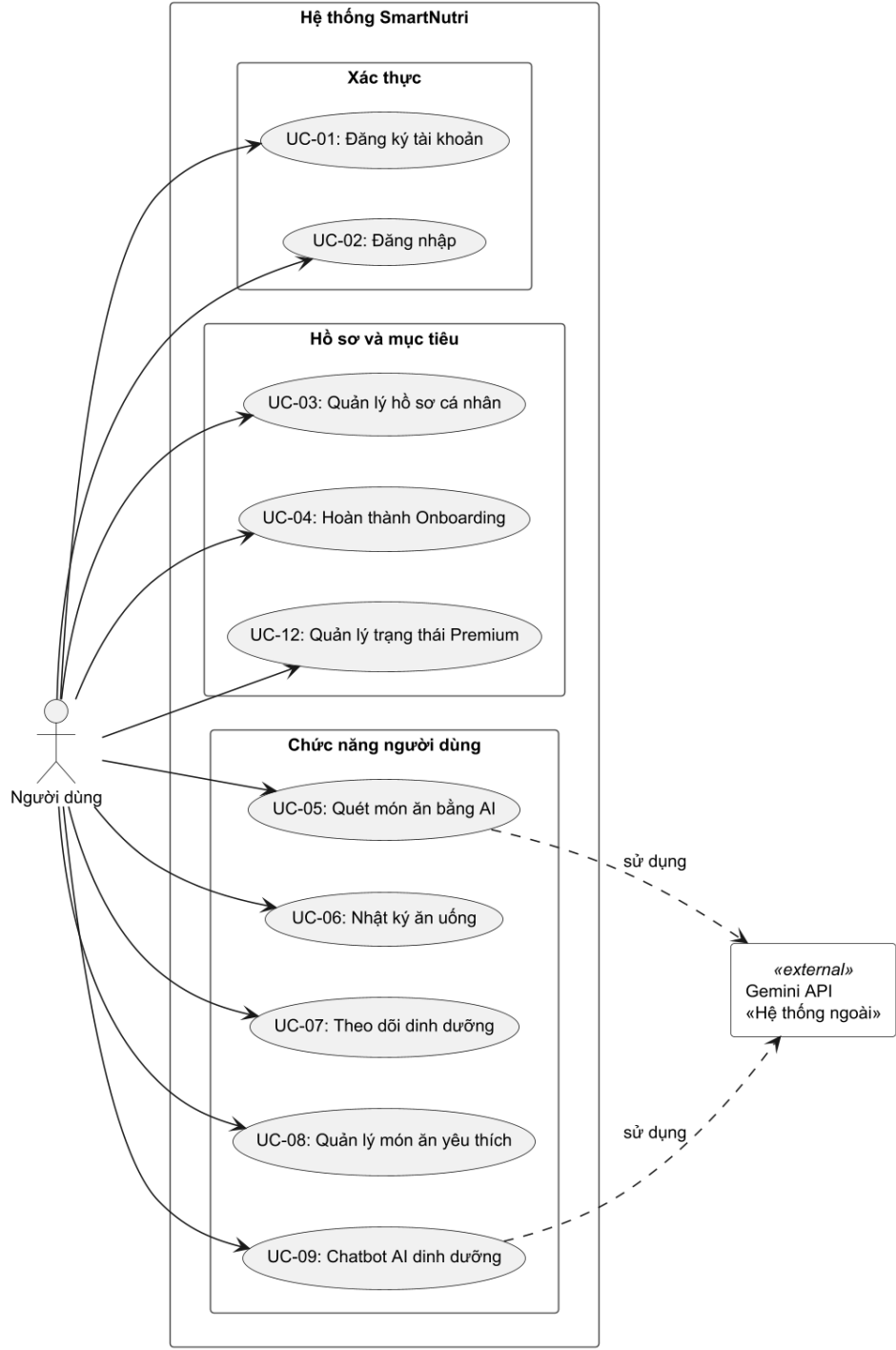
- Ứng dụng di động hỗ trợ Android 5.0 (API 21) trở lên.
- Backend phụ thuộc hoàn toàn vào hệ sinh thái Firebase; không có máy chủ backend tự quản lý.
- Gemini API có giới hạn về số request mỗi phút và độ chính xác phụ thuộc vào chất lượng ảnh đầu vào và độ phủ của dữ liệu huấn luyện đối với ẩm thực Việt Nam.
- Tính năng quét AI và chatbot yêu cầu kết nối mạng để hoạt động.

3.3 Tổng quan ca sử dụng hệ thống

Hệ thống SmartNutri có hai nhóm actor chính: **Người dùng** (người dùng cuối tương tác qua ứng dụng di động) và **Quản trị viên** (người quản lý hệ thống qua trang web). Ngoài ra, **Gemini API** đóng vai trò là hệ thống bên ngoài, được gọi từ Cloud Functions để hỗ trợ nhận diện món ăn và trả lời chatbot.

3.3.1 Biểu đồ ca sử dụng cho Người dùng

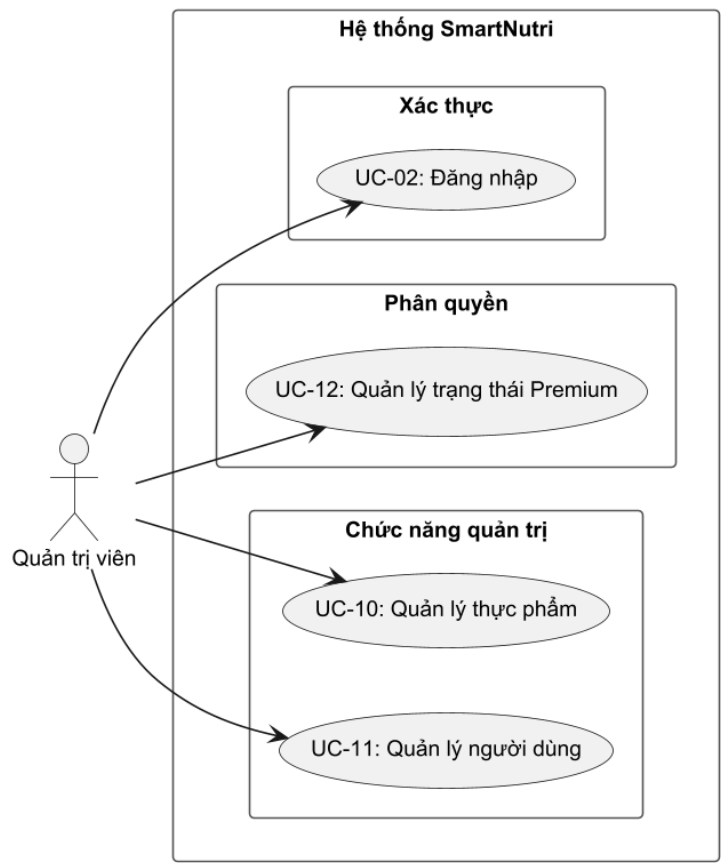
Hình 3.1 trình bày các ca sử dụng dành cho người dùng cuối. Mười ca sử dụng được chia thành ba nhóm: Xác thực (UC-01, UC-02), Hồ sơ và mục tiêu (UC-03, UC-04, UC-12) và Chức năng người dùng (UC-05, UC-06, UC-07, UC-08, UC-09). Actor Gemini API (hệ thống ngoài) hỗ trợ hai ca sử dụng AI: UC-05 (Quét món ăn bằng AI) và UC-09 (Chatbot AI dinh dưỡng), được gọi gián tiếp thông qua Cloud Functions.



Hình 3.1: Biểu đồ ca sử dụng cho Người dùng — Hệ thống SmartNutri

3.3.2 Biểu đồ ca sử dụng cho Quản trị viên

Hình 3.2 trình bày các ca sử dụng dành cho quản trị viên qua trang quản trị web. Bốn ca sử dụng được chia thành ba nhóm: Xác thực (UC-02), Phân quyền (UC-12) và Chức năng quản trị (UC-10, UC-11). Quản trị viên cần đăng nhập trước khi thực hiện các thao tác quản lý thực phẩm, quản lý người dùng và gán gói Premium.



Hình 3.2: Biểu đồ ca sử dụng cho Quản trị viên — Hệ thống SmartNutri

3.4 Mô tả ca sử dụng

3.4.1 UC-01: Đăng ký tài khoản

Thuộc tính	Mô tả
Tên ca sử dụng	Đăng ký tài khoản
Mô tả	Người dùng tạo tài khoản mới để sử dụng ứng dụng.
Actor	Người dùng (chưa có tài khoản)

Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở ứng dụng, chọn “Đăng ký”. 2. Hệ thống hiển thị form đăng ký (email, mật khẩu, xác nhận mật khẩu). 3. Người dùng nhập thông tin và nhấn “Đăng ký”. 4. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 5. Firebase Auth tạo tài khoản mới và gửi email xác minh. 6. Hệ thống tạo document người dùng trong Firestore. 7. Hệ thống chuyển đến màn hình Onboarding.
Luồng thay thế	3a. Email đã tồn tại: Hiển thị thông báo lỗi “Email đã được sử dụng”. 3b. Mật khẩu không đủ mạnh: Yêu cầu mật khẩu tối thiểu 6 ký tự. 3c. Mất kết nối mạng: Hiển thị thông báo lỗi kết nối.
Tiền điều kiện	Người dùng chưa có tài khoản trong hệ thống.
Hậu điều kiện	Tài khoản được tạo, document người dùng được lưu trong Firestore, email xác minh được gửi.
Business Rules	• Email phải đúng định dạng. • Mật khẩu tối thiểu 6 ký tự. • Mỗi email chỉ được đăng ký một tài khoản.

3.4.2 UC-02: Đăng nhập

Thuộc tính	Mô tả
Tên ca sử dụng	Đăng nhập
Mô tả	Người dùng đăng nhập vào hệ thống để truy cập các chức năng cá nhân hóa như nhật ký ăn uống, thống kê dinh dưỡng và chatbot AI.
Actor	Người dùng đã có tài khoản

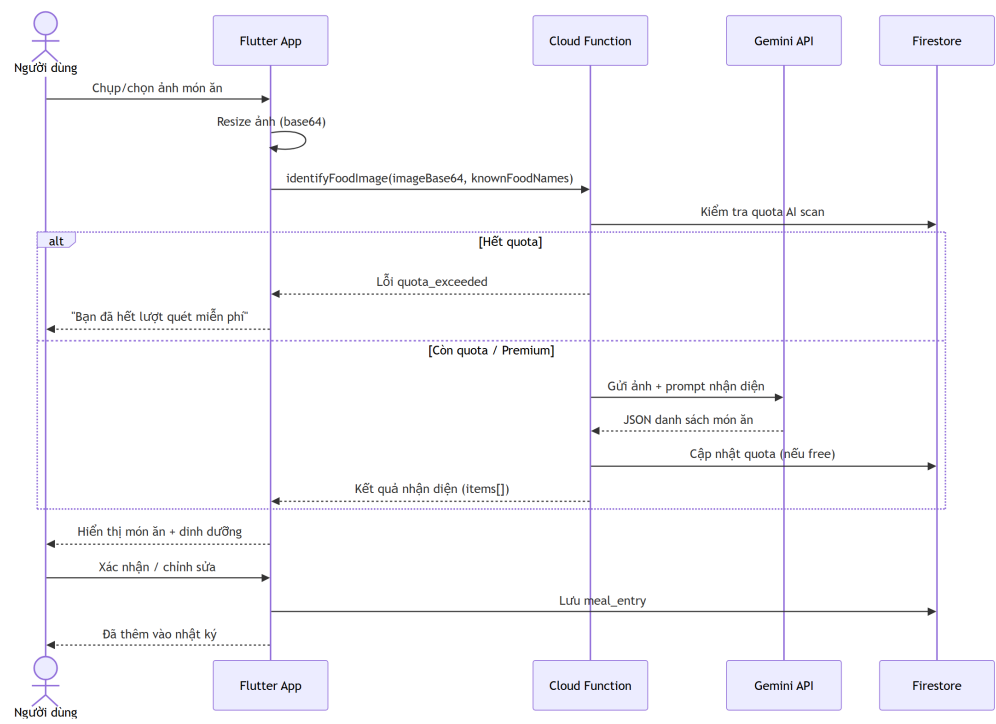
Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở ứng dụng và chọn chức năng đăng nhập. 2. Hệ thống hiển thị form đăng nhập. 3. Người dùng nhập email và mật khẩu, sau đó nhấn “Đăng nhập”. 4. Hệ thống gửi thông tin xác thực đến Firebase Authentication. 5. Firebase Authentication xác thực thông tin đăng nhập. 6. Hệ thống tải thông tin hồ sơ người dùng từ Firestore. 7. Hệ thống chuyển người dùng đến màn hình chính.
Luồng thay thế	3a. Người dùng chọn đăng nhập bằng Google: Hệ thống mở màn hình chọn tài khoản Google, sau đó xác thực qua Firebase Authentication. 4a. Email hoặc mật khẩu không hợp lệ: Hệ thống hiển thị thông báo lỗi và cho phép nhập lại. 6a. Hồ sơ người dùng chưa hoàn tất: Hệ thống chuyển người dùng đến màn hình Onboarding.
Tiền điều kiện	Người dùng đã có tài khoản trong hệ thống.
Hậu điều kiện	Người dùng được xác thực và có thể truy cập các chức năng chính của ứng dụng.
Business Rules	• Không cho phép truy cập dữ liệu cá nhân nếu người dùng chưa đăng nhập. • Thông tin xác thực được xử lý qua Firebase Authentication. • Nếu hồ sơ chưa hoàn tất, người dùng phải hoàn thành Onboarding trước khi dùng đầy đủ chức năng.

3.4.3 UC-05: Quét món ăn bằng AI

Thuộc tính	Mô tả
Tên ca sử dụng	Quét món ăn bằng AI
Mô tả	Người dùng chụp ảnh hoặc chọn ảnh món ăn từ thư viện. Ảnh được gửi lên Cloud Function, Cloud Function gọi Gemini API để nhận diện món ăn và trả về thông tin dinh dưỡng ước tính. Người dùng kiểm tra, chỉnh sửa (nếu cần) rồi xác nhận lưu vào nhật ký.

Thuộc tính	Mô tả
Actor	Người dùng đã đăng nhập
Luồng cơ bản	1. Người dùng chọn chức năng quét món ăn từ màn hình chính. 2. Hệ thống mở camera hoặc cho phép chọn ảnh từ thư viện. 3. Người dùng chụp ảnh hoặc chọn ảnh món ăn. 4. Ứng dụng resize ảnh, chuyển thành chuỗi base64 và gửi đến Cloud Function <code>identifyFoodImage</code> kèm Firebase Auth token. 5. Cloud Function xác thực token, kiểm tra quota quét AI của người dùng trong Firestore, sau đó gọi Gemini API kèm prompt phân tích và danh sách thực phẩm đã có trong cơ sở dữ liệu để ưu tiên khớp tên. 6. Gemini API trả về danh sách món ăn kèm thông tin dinh dưỡng ước tính (tên món, calo, protein, carbohydrate, chất béo, khối lượng phần ăn, độ tự tin) dưới dạng JSON. 7. Cloud Function cập nhật số lượt quét đã sử dụng trong Firestore (đối với gói free), sau đó trả kết quả về ứng dụng. 8. Ứng dụng hiển thị kết quả nhận diện cho người dùng kiểm tra. 9. Người dùng chỉnh sửa thông tin dinh dưỡng nếu cần (tên món, khối lượng, calo,...). 10. Người dùng xác nhận, hệ thống lưu món ăn vào nhật ký (<code>subcollection meal_entries</code>).

Thuộc tính	Mô tả
Luồng thay thế	3a. Ảnh không hợp lệ (dung lượng quá lớn, định dạng không hỗ trợ): Hiển thị thông báo “Ảnh không hợp lệ, vui lòng chọn ảnh khác” và cho phép chọn lại. 5a. Hết quota quét AI (gói free): Cloud Function trả về lỗi <code>quota_exceeded</code>. Ứng dụng hiển thị “Bạn đã hết lượt quét miễn phí trong tháng này” và gợi ý nâng cấp Premium hoặc nhập thủ công. 5b. Gemini API không nhận diện được món ăn (confidence dưới ngưỡng): Cloud Function trả về danh sách rỗng. Ứng dụng hiển thị “Không thể nhận diện món ăn, vui lòng thử lại với ảnh khác”. 5c. Lỗi mạng hoặc Gemini API lỗi/quá tải: Cloud Function trả về lỗi. Ứng dụng hiển thị “Đã xảy ra lỗi, vui lòng thử lại sau” và cho phép người dùng thử lại hoặc nhập thủ công. 3b/8a. Người dùng hủy thao tác: Quay về màn hình trước đó (màn hình chính hoặc màn hình nhật ký).
Tiền điều kiện	Người dùng đã đăng nhập; thiết bị có kết nối mạng.
Hậu điều kiện	Món ăn được lưu vào nhật ký dinh dưỡng của người dùng trong ngày hiện tại (subcollection <code>users/{uid}/meal_entries</code>). Đối với gói free, số lượt quét AI trong tháng được tăng lên 1.
Business Rules	- Kết quả dinh dưỡng từ AI chỉ mang tính tham khảo; người dùng phải được xem lại và có quyền chỉnh sửa trước khi lưu vào nhật ký. - Gemini API không được gọi trực tiếp từ ứng dụng di động. Toàn bộ request đến Gemini phải đi qua Cloud Function để bảo vệ API key. - Người dùng gói free bị giới hạn 5 lượt quét AI mỗi tháng. Người dùng gói Premium không bị giới hạn. - Ảnh gửi lên phải có định dạng JPEG hoặc PNG, dung lượng không vượt quá giới hạn của Cloud Function request body.

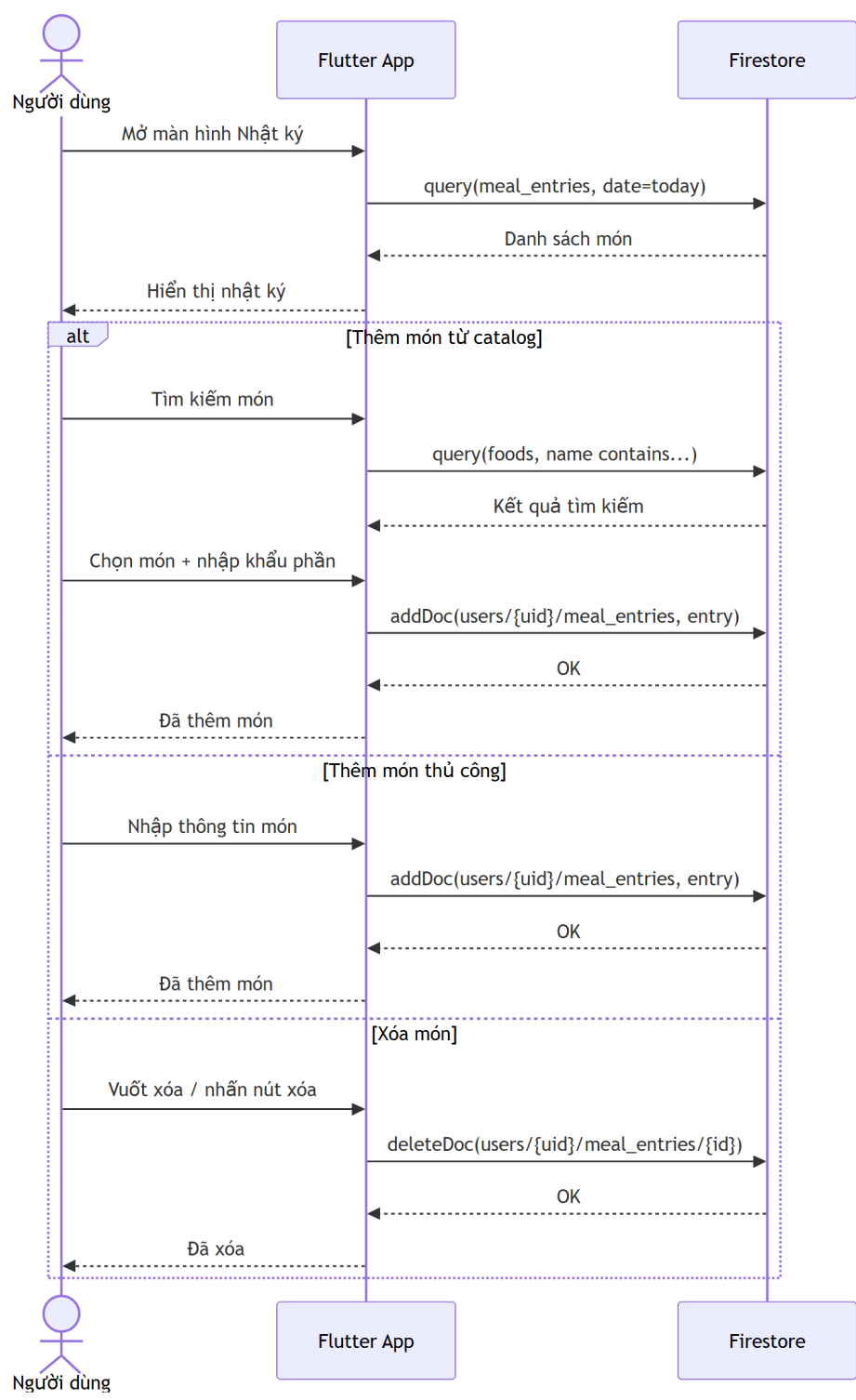


Hình 3.3: Biểu đồ tuần tự ca sử dụng Quét món ăn bằng AI

3.4.4 UC-06: Nhật ký ăn uống

Thuộc tính	Mô tả
Tên ca sử dụng	Nhật ký ăn uống
Mô tả	Người dùng xem, thêm, chỉnh sửa hoặc xóa các món ăn đã ghi nhận trong ngày để theo dõi lượng dinh dưỡng tiêu thụ.
Actor	Người dùng đã đăng nhập

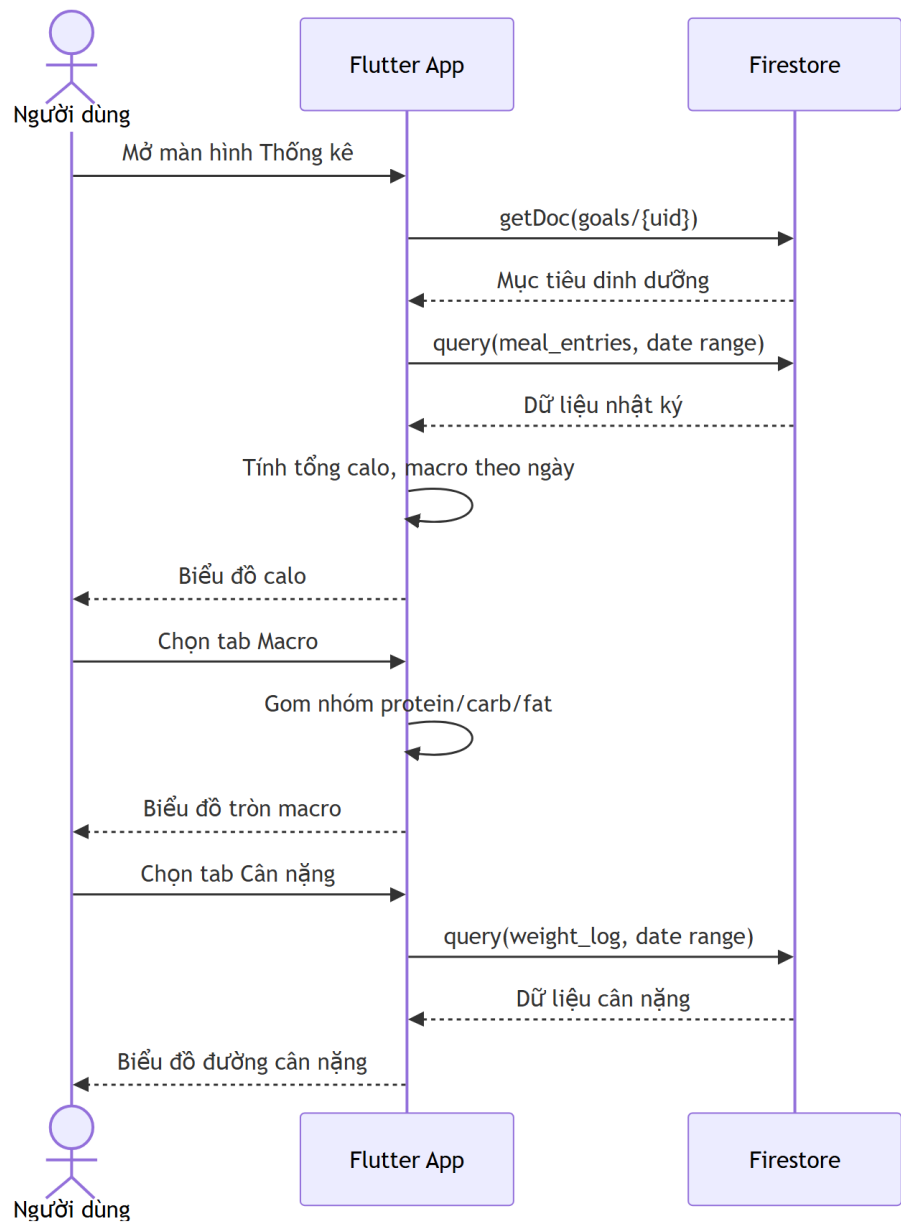
Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở màn hình nhật ký ăn uống. 2. Hệ thống truy xuất danh sách món ăn đã ghi nhận trong ngày từ Firestore. 3. Hệ thống hiển thị danh sách món ăn theo từng bữa: sáng, trưa, tối và bữa phụ. 4. Người dùng chọn thêm món ăn thủ công hoặc thêm từ kết quả quét AI. 5. Người dùng nhập khẩu phần hoặc điều chỉnh thông tin dinh dưỡng nếu cần. 6. Hệ thống lưu bản ghi mới vào sub-collection nhật ký của người dùng. 7. Hệ thống cập nhật tổng calo và các chỉ số dinh dưỡng trong ngày.
Luồng thay thế	2a. Không có dữ liệu trong ngày: Hệ thống hiển thị trạng thái rỗng và gợi ý người dùng thêm món ăn. 5a. Người dùng hủy thao tác thêm món: Hệ thống quay lại danh sách nhật ký hiện tại. 6a. Lưu dữ liệu thất bại do mất mạng: Hệ thống hiển thị thông báo lỗi và cho phép thử lại.
Tiền điều kiện	Người dùng đã đăng nhập và có quyền truy cập dữ liệu cá nhân của mình.
Hậu điều kiện	Nhật ký ăn uống được cập nhật, tổng dinh dưỡng trong ngày được tính lại.
Business Rules	• Mỗi bản ghi nhật ký thuộc về đúng một người dùng. • Người dùng chỉ được chỉnh sửa hoặc xóa bản ghi của chính mình. • Thông tin dinh dưỡng được tính theo khẩu phần thực tế đã nhập.



Hình 3.4: Biểu đồ tuần tự ca sử dụng Nhật ký ăn uống

3.4.5 UC-07: Theo dõi dinh dưỡng

Thuộc tính	Mô tả
Tên ca sử dụng	Theo dõi dinh dưỡng
Mô tả	Người dùng xem tổng quan lượng calo và các nhóm chất dinh dưỡng đã tiêu thụ, từ đó so sánh với mục tiêu cá nhân.
Actor	Người dùng đã đăng nhập
Luồng cơ bản	1. Người dùng mở màn hình thông kê dinh dưỡng. 2. Hệ thống truy xuất nhật ký ăn uống theo khoảng thời gian được chọn. 3. Hệ thống tính tổng calo, protein, carbohydrate và chất béo. 4. Hệ thống so sánh kết quả với mục tiêu dinh dưỡng của người dùng. 5. Hệ thống hiển thị biểu đồ và thẻ thống kê tổng quan. 6. Người dùng chuyển đổi giữa chế độ xem ngày, tuần hoặc tháng.
Luồng thay thế	2a. Không có dữ liệu trong khoảng thời gian được chọn: Hệ thống hiển thị thông báo chưa có dữ liệu. 4a. Người dùng chưa thiết lập mục tiêu dinh dưỡng: Hệ thống hiển thị số liệu tiêu thụ và gợi ý hoàn thiện hồ sơ. 6a. Người dùng chọn khoảng thời gian khác: Hệ thống tải lại dữ liệu tương ứng.
Tiền điều kiện	Người dùng đã đăng nhập; dữ liệu nhật ký ăn uống có thể tồn tại hoặc chưa tồn tại.
Hậu điều kiện	Người dùng nắm được tình trạng tiêu thụ dinh dưỡng so với mục tiêu cá nhân.
Business Rules	• Số liệu thống kê chỉ tính từ dữ liệu nhật ký của người dùng hiện tại. • Các chỉ số macro bao gồm protein, carbohydrate và chất béo. • Nếu dữ liệu thiếu hoặc chưa đủ, hệ thống phải thể hiện rõ trạng thái chưa có dữ liệu thay vì suy đoán.



Hình 3.5: Biểu đồ tuần tự ca sử dụng Theo dõi dinh dưỡng

3.4.6 UC-03: Quản lý hồ sơ cá nhân

Thuộc tính	Mô tả
Tên ca sử dụng	Quản lý hồ sơ cá nhân
Mô tả	Người dùng xem và cập nhật thông tin cá nhân như họ tên, giới tính, chiều cao, cân nặng, độ tuổi và các thông tin phục vụ tính toán mục tiêu dinh dưỡng.
Actor	Người dùng đã đăng nhập

Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở màn hình hồ sơ cá nhân. 2. Hệ thống tải thông tin hồ sơ hiện tại từ Firestore. 3. Người dùng chỉnh sửa các trường thông tin cần cập nhật. 4. Người dùng nhấn nút lưu thay đổi. 5. Hệ thống kiểm tra dữ liệu đầu vào. 6. Hệ thống cập nhật document hồ sơ người dùng trong Firestore. 7. Hệ thống hiển thị thông báo cập nhật thành công.
Luồng thay thế	2a. Chưa có hồ sơ: Hệ thống hiển thị form rỗng và yêu cầu người dùng bổ sung thông tin. 5a. Dữ liệu không hợp lệ: Hệ thống hiển thị lỗi tại trường tương ứng. 6a. Lưu thất bại do lỗi mạng: Hệ thống thông báo lỗi và cho phép thử lại.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống.
Hậu điều kiện	Thông tin hồ sơ cá nhân được cập nhật và có thể được sử dụng cho các tính năng tính toán mục tiêu dinh dưỡng.
Business Rules	• Người dùng chỉ được cập nhật hồ sơ của chính mình. • Các trường như chiều cao, cân nặng và tuổi phải là giá trị hợp lệ. • Dữ liệu hồ sơ là cơ sở để tính toán mục tiêu calo và gợi ý dinh dưỡng.

3.4.7 UC-04: Hoàn thành Onboarding

Thuộc tính	Mô tả
Tên ca sử dụng	Hoàn thành Onboarding
Mô tả	Người dùng mới cung cấp các thông tin ban đầu để hệ thống thiết lập hồ sơ cá nhân và mục tiêu dinh dưỡng phù hợp.
Actor	Người dùng mới đăng ký hoặc chưa hoàn thiện hồ sơ

Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng đăng nhập lần đầu hoặc được chuyển đến màn hình Onboarding. 2. Hệ thống hiển thị các bước nhập thông tin cơ bản. 3. Người dùng nhập tuổi, giới tính, chiều cao, cân nặng và mục tiêu sức khỏe. 4. Hệ thống tính toán mục tiêu dinh dưỡng ban đầu dựa trên thông tin đã nhập. 5. Người dùng xác nhận thông tin. 6. Hệ thống lưu hồ sơ và mục tiêu dinh dưỡng vào Firestore. 7. Hệ thống chuyển người dùng đến màn hình chính.
Luồng thay thế	3a. Người dùng nhập thiếu thông tin bắt buộc: Hệ thống yêu cầu hoàn thiện trước khi tiếp tục. 4a. Không thể tính mục tiêu do dữ liệu không hợp lệ: Hệ thống yêu cầu kiểm tra lại thông tin. 6a. Lưu thất bại: Hệ thống hiển thị thông báo lỗi và cho phép lưu lại.
Tiền điều kiện	Người dùng đã đăng nhập nhưng chưa hoàn tất hồ sơ ban đầu.
Hậu điều kiện	Hồ sơ người dùng và mục tiêu dinh dưỡng ban đầu được tạo trong hệ thống.
Business Rules	• Người dùng phải hoàn thành các thông tin bắt buộc trước khi sử dụng đầy đủ ứng dụng. • Mục tiêu dinh dưỡng ban đầu có thể được chỉnh sửa lại sau trong phần hồ sơ hoặc mục tiêu. • Dữ liệu Onboarding phải gắn với đúng tài khoản đang đăng nhập.

3.4.8 UC-08: Quản lý món ăn yêu thích

Thuộc tính	Mô tả
Tên ca sử dụng	Quản lý món ăn yêu thích
Mô tả	Người dùng thêm hoặc xóa món ăn khỏi danh sách yêu thích để có thể truy cập và thêm nhanh các món thường dùng.
Actor	Người dùng đã đăng nhập

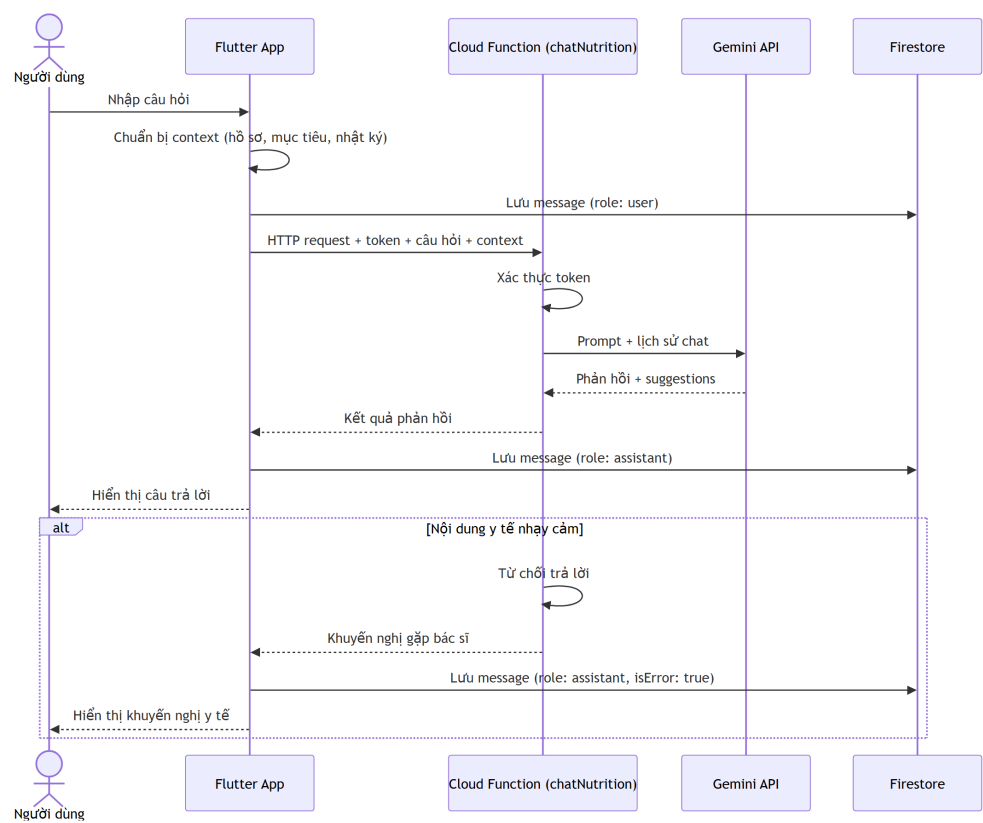
Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở danh sách món ăn hoặc màn hình tìm kiếm thực phẩm. 2. Hệ thống hiển thị trạng thái yêu thích của từng món ăn. 3. Người dùng nhấn biểu tượng yêu thích trên một món ăn. 4. Hệ thống kiểm tra trạng thái hiện tại của món ăn. 5. Nếu món chưa được yêu thích, hệ thống ghi bản ghi vào ‘users/{uid}/favorites’. 6. Nếu món đã được yêu thích, hệ thống xóa bản ghi tương ứng. 7. Hệ thống cập nhật lại giao diện danh sách yêu thích.
Luồng thay thế	2a. Không có món yêu thích: Hệ thống hiển thị trạng thái rỗng và gợi ý người dùng thêm món. 5a. Món ăn không còn tồn tại trong food catalog: Hệ thống bỏ qua hoặc không hiển thị món trong danh sách yêu thích. 6a. Lỗi ghi/xóa Firestore: Hệ thống thông báo lỗi và giữ nguyên trạng thái trước đó.
Tiền điều kiện	Người dùng đã đăng nhập và food catalog đã được tải.
Hậu điều kiện	Danh sách món ăn yêu thích của người dùng được cập nhật.
Business Rules	• Mỗi món ăn yêu thích được định danh theo ‘foodId’. • Người dùng chỉ đọc/ghi danh sách yêu thích của chính mình. • Danh sách yêu thích phải đồng bộ giữa màn hình Home và màn hình Search.

3.4.9 UC-09: Chatbot AI dinh dưỡng

Thuộc tính	Mô tả
Tên ca sử dụng	Chatbot AI dinh dưỡng
Mô tả	Người dùng đặt câu hỏi về dinh dưỡng, chế độ ăn uống và món ăn. Hệ thống sử dụng Gemini API để phản hồi ở mức tham khảo, dựa trên ngữ cảnh cá nhân của người dùng (hồ sơ, mục tiêu dinh dưỡng, nhật ký ăn uống gần đây).
Actor	Người dùng đã đăng nhập

Thuộc tính	Mô tả
Luồng cơ bản	1. Người dùng mở màn hình chatbot từ thanh điều hướng. 2. Hệ thống tải lịch sử trò chuyện gần nhất từ <code>users/{uid}/chat_history</code> và hiển thị. Nếu chưa có lịch sử, hiển thị tin nhắn chào mừng. 3. Người dùng nhập câu hỏi và nhấn gửi. 4. Ứng dụng kiểm tra câu hỏi không rỗng, sau đó lưu tin nhắn của người dùng vào <code>chat_history</code> với <code>role: user</code>. 5. Ứng dụng xây dựng ngữ cảnh từ hồ sơ cá nhân, mục tiêu dinh dưỡng và nhật ký ăn uống gần đây của người dùng. 6. Ứng dụng gửi câu hỏi kèm ngữ cảnh đến Cloud Function <code>chatNutrition</code> qua HTTP request có kèm Firebase Auth token. 7. Cloud Function xác thực token, gọi Gemini API với prompt được thiết kế để giới hạn phạm vi trả lời trong lĩnh vực dinh dưỡng. 8. Gemini API trả về nội dung phản hồi và gợi ý câu hỏi tiếp theo. 9. Cloud Function trả kết quả về ứng dụng. Ứng dụng lưu tin nhắn phản hồi vào <code>chat_history</code> với <code>role: assistant</code>. 10. Ứng dụng hiển thị phản hồi cho người dùng.

Thuộc tính	Mô tả
Luồng thay thế	3a. Câu hỏi trống: Ứng dụng hiển thị thông báo “Vui lòng nhập câu hỏi” và không gửi request. 6a. Lỗi mạng hoặc Cloud Function không khả dụng: Ứng dụng hiển thị “Không thể kết nối, vui lòng thử lại sau” và cho phép người dùng thử lại. 7a. Gemini API không phản hồi hoặc trả về lỗi: Cloud Function trả về lỗi. Ứng dụng hiển thị “Trợ lý AI hiện không khả dụng, vui lòng thử lại sau”. Tin nhắn lỗi được lưu vào chat_history với isError: true để người dùng biết tin nhắn này không nhận được phản hồi. 7b. Người dùng gửi câu hỏi liên quan đến chẩn đoán y tế hoặc nội dung nhạy cảm: Gemini được cấu hình qua prompt để từ chối trả lời và khuyến nghị người dùng tham khảo ý kiến bác sĩ hoặc chuyên gia y tế. 3b. Người dùng đóng màn hình chatbot: Quay về màn hình trước đó.
Tiền điều kiện	Người dùng đã đăng nhập; thiết bị có kết nối mạng; Gemini API khả dụng.
Hậu điều kiện	Cập tin nhắn (user + assistant hoặc user + error) được lưu vào users/{uid}/chat_history. Phản hồi được hiển thị trên giao diện chat.
Business Rules	- Toàn bộ câu trả lời từ chatbot chỉ mang tính tham khảo về dinh dưỡng, không thay thế tư vấn của bác sĩ hoặc chuyên gia dinh dưỡng. - Chatbot không được đưa ra chẩn đoán y tế, kê đơn thuốc, hoặc khuyến nghị điều trị bệnh. Đối với các câu hỏi mang tính y tế nghiêm trọng, chatbot phải khuyến nghị người dùng tìm đến cơ sở y tế. - Gemini API được gọi thông qua Cloud Function, không gọi trực tiếp từ ứng dụng di động, nhằm bảo vệ API key. - Lịch sử trò chuyện được lưu riêng theo từng người dùng (phân tách theo UID) và chỉ chủ sở hữu mới có quyền truy cập.



Hình 3.6: Biểu đồ tuần tự ca sử dụng Chatbot AI dinh dưỡng

3.4.10 UC-10: Quản lý thực phẩm

Thuộc tính	Mô tả
Tên ca sử dụng	Quản lý thực phẩm
Mô tả	Quản trị viên quản lý dữ liệu thực phẩm trong food catalog, bao gồm xem danh sách, tìm kiếm, thêm mới, chỉnh sửa hoặc vô hiệu hóa món ăn.
Actor	Quản trị viên
Luồng cơ bản	1. Quản trị viên đăng nhập vào trang quản trị. 2. Hệ thống xác thực quyền quản trị. 3. Quản trị viên mở màn hình quản lý thực phẩm. 4. Hệ thống hiển thị danh sách thực phẩm từ collection ‘foods‘. 5. Quản trị viên thực hiện thao tác thêm, sửa hoặc vô hiệu hóa thực phẩm. 6. Hệ thống kiểm tra tính hợp lệ của dữ liệu dinh dưỡng. 7. Hệ thống ghi thay đổi vào Firestore. 8. Hệ thống cập nhật lại danh sách thực phẩm trên giao diện.

Thuộc tính	Mô tả
Luồng thay thế	2a. Tài khoản không có quyền admin: Hệ thống từ chối truy cập. 6a. Thiếu trường bắt buộc hoặc giá trị dinh dưỡng không hợp lệ: Hệ thống hiển thị lỗi nhập liệu. 7a. Ghi dữ liệu thất bại: Hệ thống thông báo lỗi và không cập nhật giao diện thành công.
Tiền điều kiện	Quản trị viên đã đăng nhập và có quyền quản trị.
Hậu điều kiện	Dữ liệu food catalog được cập nhật theo thao tác hợp lệ của quản trị viên.
Business Rules	- Chỉ tài khoản có quyền admin mới được ghi vào collection ‘foods’. - Thông tin dinh dưỡng phải có các chỉ số cơ bản như calo, protein, carbohydrate và chất béo. - Không nên xóa cứng dữ liệu đang được tham chiếu bởi nhật ký người dùng; ưu tiên vô hiệu hóa nếu cần ẩn món ăn.

3.4.11 UC-11: Quản lý người dùng

Thuộc tính	Mô tả
Tên ca sử dụng	Quản lý người dùng
Mô tả	Quản trị viên xem danh sách người dùng, theo dõi thông tin cơ bản và trạng thái sử dụng để phục vụ công tác quản trị hệ thống.
Actor	Quản trị viên
Luồng cơ bản	- Quản trị viên đăng nhập vào trang quản trị. - Hệ thống xác thực quyền quản trị. - Quản trị viên mở màn hình quản lý người dùng. - Hệ thống truy xuất danh sách người dùng từ collection ‘users’. - Quản trị viên tìm kiếm hoặc lọc danh sách theo thông tin cần xem. - Hệ thống hiển thị thông tin hồ sơ, trạng thái gói dịch vụ và usage liên quan nếu có.

Thuộc tính	Mô tả
Luồng thay thế	2a. Tài khoản không có quyền admin: Hệ thống từ chối truy cập. 4a. Không tải được danh sách người dùng: Hệ thống hiển thị thông báo lỗi. 5a. Không có kết quả tìm kiếm phù hợp: Hệ thống hiển thị trạng thái rỗng.
Tiền điều kiện	Quản trị viên đã đăng nhập và có quyền truy cập trang quản trị.
Hậu điều kiện	Quản trị viên xem được thông tin người dùng phục vụ theo dõi và quản trị.
Business Rules	- Thông tin người dùng chỉ được hiển thị cho tài khoản có quyền admin. - Các dữ liệu nhạy cảm như mật khẩu không được lưu hoặc hiển thị trong Firestore. - Các thao tác thay đổi trạng thái người dùng cần được kiểm soát quyền chặt chẽ.

3.4.12 UC-12: Quản lý trạng thái Premium

Thuộc tính	Mô tả
Tên ca sử dụng	Quản lý trạng thái Premium
Mô tả	Người dùng xem trạng thái gói tài khoản hiện tại (Free hoặc Premium). Hệ thống phân biệt giữa hai gói để giới hạn hoặc mở khóa các chức năng tương ứng (ví dụ: số lượt quét AI mỗi tháng). Trong phạm vi hiện tại của dự án, việc gán Premium được thực hiện thông qua Cloud Function bởi quản trị viên ở mức mô phỏng; tích hợp thanh toán thật qua Google Play Billing là định hướng phát triển trong tương lai.
Actor	Người dùng đã đăng nhập; Quản trị viên (cho thao tác gán Premium)

Thuộc tính	Mô tả
Luồng cơ bản (phía người dùng)	1. Người dùng mở màn hình Premium hoặc màn hình hồ sơ. 2. Hệ thống đọc thông tin subscription từ document <code>users/{uid}</code>. 3. Hệ thống hiển thị trạng thái gói hiện tại (Free hoặc Premium), kèm các quyền lợi tương ứng. 4. Nếu là Premium, hiển thị thời hạn (nếu có) và danh sách tính năng đã mở khóa.
Luồng cơ bản (phía quản trị viên)	1. Quản trị viên đăng nhập vào trang admin. 2. Quản trị viên chọn người dùng cần gán Premium từ danh sách. 3. Quản trị viên chọn gói premium, thiết lập thời hạn (nếu có) và nhấn xác nhận. 4. Trang admin gọi Cloud Function <code>setUserSubscription</code> với tham số <code>targetUid</code>, <code>plan</code>, <code>premiumUntil</code>. 5. Cloud Function xác thực quyền admin, cập nhật trường <code>subscription</code> trong document <code>users/{uid}</code> với <code>plan: premium, status: active, source: admin</code>. 6. Hệ thống cập nhật giao diện admin và mở khóa quyền lợi Premium cho người dùng được gán.
Luồng thay thế	2a. Người dùng chưa có thông tin subscription: Hệ thống mặc định hiển thị gói Free. 3a. Premium đã hết hạn (<code>premiumUntil < hiện tại</code>): Hệ thống hiển thị gói Free và thông báo gói Premium đã hết hạn. 4a. (Admin) Người dùng được gán lại gói Free: Cloud Function cập nhật <code>plan: free, status: none</code>. Các giới hạn của gói Free được áp dụng trở lại. 5a. (Admin) Token không có admin claim: Cloud Function từ chối với lỗi <code>not_an_admin</code>.
Tiền điều kiện	Người dùng đã đăng nhập; quản trị viên có tài khoản với admin claim.
Hậu điều kiện	Trạng thái gói của người dùng được hiển thị chính xác trên cả ứng dụng di động và trang admin. Nếu admin thay đổi gói, document <code>users/{uid}</code> được cập nhật và ứng dụng di động phản ánh trạng thái mới trong lần kiểm tra tiếp theo.

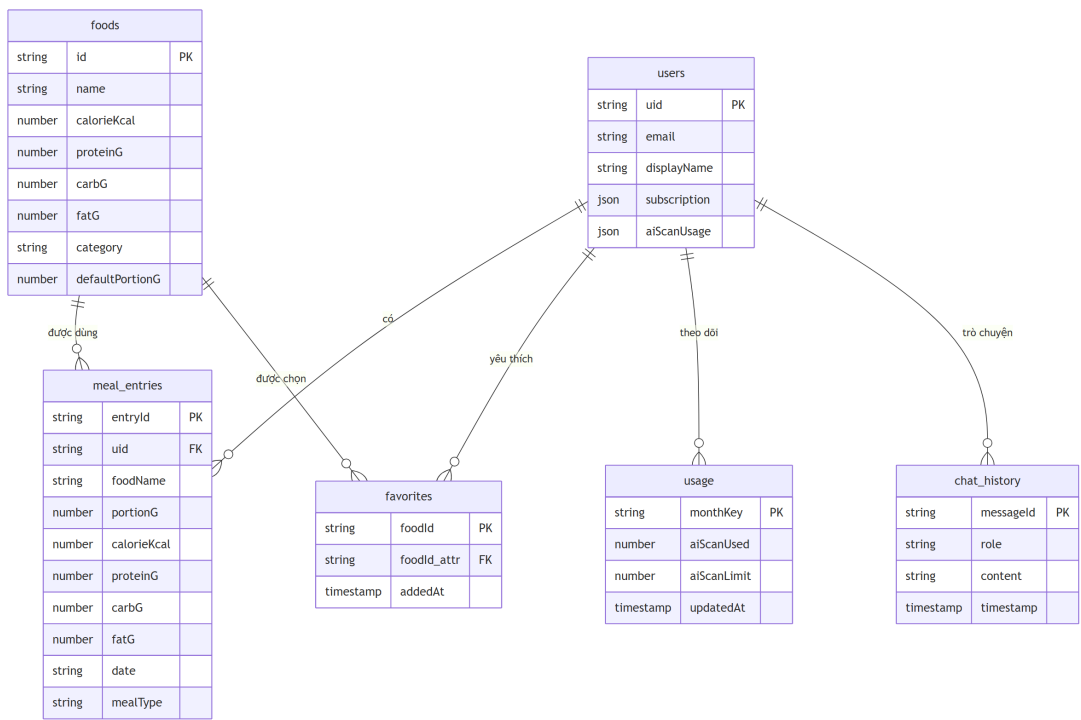
Thuộc tính	Mô tả
Business Rules	- Hệ thống phân biệt hai gói: Free (mặc định) và Premium. Gói Free giới hạn 5 lượt quét AI mỗi tháng; gói Premium không giới hạn lượt quét. - Trạng thái subscription được lưu trong trường subscription của document users/{uid}, gồm các thuộc tính: plan (free/premium), status (none/active), source (default/admin/mock), premiumUntil (tùy chọn), updatedAt. - Trong phạm vi dự án hiện tại, việc nâng cấp lên Premium được thực hiện thông qua Cloud Function bởi quản trị viên ở mức mô phỏng, chưa tích hợp thanh toán thật. - Định hướng phát triển: tích hợp Google Play Billing để người dùng có thể tự đăng ký và thanh toán Premium ngay trong ứng dụng; khi đó trường source sẽ được đặt là iap thay vì admin hay mock.

3.5 Thiết kế cơ sở dữ liệu

Hệ thống SmartNutri sử dụng **Cloud Firestore** — cơ sở dữ liệu NoSQL dạng document của Firebase. Khác với cơ sở dữ liệu quan hệ (RDBMS), Firestore không sử dụng bảng với schema cố định. Dữ liệu được tổ chức thành các **collection** (tập hợp document), mỗi **document** là một bản ghi JSON có cấu trúc linh hoạt. Một document có thể chứa các **sub-collection** — tập hợp document con nằm bên trong nó, tạo thành cấu trúc phân cấp.

3.5.1 Về sơ đồ ERD

Sơ đồ dưới đây (Hình 3.7) mô tả **quan hệ logic** giữa các nhóm dữ liệu trong hệ thống. Do Firestore là NoSQL, sơ đồ này không biểu diễn quan hệ khóa ngoại như trong cơ sở dữ liệu quan hệ. Các mũi tên trong sơ đồ thể hiện mối liên kết dữ liệu được thực thi ở tầng ứng dụng hoặc thông qua Firestore Security Rules, không phải ràng buộc ở tầng cơ sở dữ liệu.



Hình 3.7: Sơ đồ quan hệ logic giữa các nhóm dữ liệu trong SmartNutri

3.5.2 Nguyên tắc phân tách dữ liệu người dùng

Toàn bộ dữ liệu cá nhân của người dùng được tổ chức dưới đường dẫn `users/{userId}/`. Các sub-collection như `meal_entries`, `favorites`, `usage`, `chat_history` đều nằm trong document của từng người dùng. Firestore Security Rules đảm bảo mỗi người dùng chỉ có quyền đọc/ghi dữ liệu thuộc `users/{userId}/` của chính mình, trong đó `{userId}` được xác định từ Firebase Authentication token.

3.5.3 Mô tả các collection chính

3.5.3.1 Collection: users

Collection gốc `users` lưu trữ thông tin tài khoản của từng người dùng. Mỗi document được định danh bằng UID từ Firebase Authentication. Document này cũng là nơi chứa các sub-collection cho dữ liệu cá nhân.

Trường	Kiểu dữ liệu	Mô tả
email	String	Email đăng ký của người dùng
displayName	String	Tên hiển thị (có thể trống)
mealCount	Number	Tổng số bữa ăn đã ghi nhận

Trường	Kiểu dữ liệu	Mô tả
lastMealDate	String	Ngày ghi nhận bữa ăn gần nhất (yyyy-MM-dd)
subscription.plan	String	Gói dịch vụ: free (mặc định) hoặc premium
subscription.status	String	Trạng thái gói: none, active
subscription.source	String	Nguồn gán gói: default, admin, mock, iap
subscription.premiumUntil	Timestamp	Thời điểm hết hạn Premium (nếu có)
subscription.updatedAt	Timestamp	Thời điểm cập nhật subscription gần nhất
aiScanUsage.monthKey	String	Tháng áp dụng quota quét AI (yyyy-MM)
aiScanUsage.aiScanUsed	Number	Số lượt quét AI đã sử dụng trong tháng
aiScanUsage.aiScanLimit	Number	Giới hạn lượt quét AI trong tháng
onboardingCompleted	Boolean	Đánh dấu đã hoàn tất onboarding
createdAt	Timestamp	Thời điểm tạo tài khoản

Ghi chú: Thông tin hồ sơ cá nhân (tuổi, giới tính, chiều cao, cân nặng, mức vận động) được lưu trong collection riêng `profiles/{userId}`. Mục tiêu dinh dưỡng (calo, protein, carbohydrate, chất béo) được lưu trong collection `goals/{userId}`. Việc tách riêng này giúp cập nhật hồ sơ và mục tiêu độc lập mà không cần ghi lại toàn bộ document người dùng.

3.5.3.2 Collection: foods

Collection `foods` lưu trữ danh mục thực phẩm và món ăn — đây là dữ liệu dùng chung cho toàn bộ người dùng. Mỗi document đại diện cho một thực phẩm hoặc món ăn với thông tin dinh dưỡng chuẩn trên 100g.

Trường	Kiểu dữ liệu	Mô tả
id	String	Mã định danh thực phẩm (document ID)
name	String	Tên món ăn (tiếng Việt có dấu)
category	String	Danh mục (cơm, phở, canh, thịt, rau,...)
calorieKcal	Number	Năng lượng (kcal/100g)
proteinG	Number	Protein (g/100g)
carbG	Number	Carbohydrate (g/100g)
fatG	Number	Chất béo (g/100g)
defaultPortionG	Number	Khẩu phần mặc định (gram)
tags	Array<String>	Từ khóa tìm kiếm
imageUrl	String	URL hình ảnh món ăn (nếu có)

Trường	Kiểu dữ liệu	Mô tả
region	String	Vùng miền (Bắc/Trung/Nam, tùy chọn)
brand	String	Thương hiệu (tùy chọn)
verified	Boolean	Đã được kiểm duyệt bởi admin
createdAt	Timestamp	Thời điểm tạo
updatedAt	Timestamp	Thời điểm cập nhật gần nhất

Ghi chú: Người dùng đã xác thực có quyền đọc toàn bộ collection foods. Chỉ quản trị viên (có admin claim) mới có quyền ghi. Trường `verified` giúp phân biệt thực phẩm đã được kiểm duyệt với thực phẩm mới thêm/chưa duyệt.

3.5.3.3 Sub-collection: `users/{userId}/meal_entries`

Lưu trữ nhật ký ăn uống hằng ngày của từng người dùng. Mỗi document là một bản ghi món ăn được thêm vào một bữa cụ thể trong ngày.

Trường	Kiểu dữ liệu	Mô tả
uid	String	ID người dùng sở hữu bản ghi
date	String	Ngày ghi nhận (yyyy-MM-dd)
mealType	String	Loại bữa ăn (breakfast, lunch, dinner, snack)
foodName	String	Tên món ăn hoặc thực phẩm
portionG	Number	Khối lượng khẩu phần (gram)
calorieKcal	Number	Năng lượng (kcal)
proteinG	Number	Protein (g)
carbG	Number	Carbohydrate (g)
fatG	Number	Chất béo (g)
loggedAt	Timestamp	Thời điểm ghi nhận

3.5.3.4 Sub-collection: `users/{userId}/favorites`

Lưu trữ danh sách món ăn yêu thích của từng người dùng. Document ID chính là `foodId` của món ăn, giúp kiểm tra nhanh một món đã được yêu thích hay chưa.

Trường	Kiểu dữ liệu	Mô tả
foodId	String	ID của món ăn trong collection foods
addedAt	Timestamp	Thời điểm thêm vào danh sách yêu thích

3.5.3.5 Sub-collection: users/{userId}/usage

Theo dõi số lượt sử dụng tính năng quét AI theo từng tháng, phục vụ giới hạn quota cho người dùng gói Free.

Trường	Kiểu dữ liệu	Mô tả
monthKey	String	Khóa tháng (yyyy-MM) — cũng là document ID
aiScanUsed	Number	Số lượt quét AI đã sử dụng trong tháng
aiScanLimit	Number	Giới hạn lượt quét AI trong tháng (mặc định: 5)
updatedAt	Timestamp	Thời điểm cập nhật gần nhất

3.5.3.6 Sub-collection: users/{userId}/chat_history

Lưu trữ lịch sử trò chuyện giữa người dùng và chatbot AI dinh dưỡng, dùng để hiển thị lại đoạn hội thoại và cung cấp ngữ cảnh cho các lượt chat tiếp theo.

Trường	Kiểu dữ liệu	Mô tả
role	String	Vai trò người gửi: user hoặc assistant
content	String	Nội dung tin nhắn
suggestions	Array<String>	Gợi ý câu hỏi tiếp theo (từ assistant)
isError	Boolean	Đánh dấu tin nhắn lỗi (true nếu AI không phản hồi)
timestamp	Timestamp	Thời điểm ghi nhận tin nhắn

CHƯƠNG 4

Xây dựng, triển khai và kiểm thử hệ thống

4.1 Cài đặt hệ thống

4.1.1 Môi trường phát triển

Dự án SmartNutri được phát triển trên môi trường Windows với các công cụ chính sau:

- **IDE:** Visual Studio Code và Android Studio — dùng để viết code Flutter/Dart, React/TypeScript và quản lý Android Emulator.
- **Flutter SDK:** Phiên bản 3.x, ngôn ngữ Dart — phát triển ứng dụng di động.
- **Node.js:** Phiên bản 20 LTS — môi trường runtime cho Cloud Functions và Admin Web.
- **Firebase CLI:** Công cụ dòng lệnh để khởi tạo project, deploy Cloud Functions, Firestore Rules, Storage Rules và Hosting.
- **Git & GitHub:** Quản lý mã nguồn và phiên bản.
- **Android Emulator / Thiết bị Android thật:** Môi trường chạy và kiểm thử ứng dụng trong quá trình phát triển.

4.1.2 Cài đặt ứng dụng di động (Flutter)

Ứng dụng di động SmartNutri được phát triển bằng Flutter. Các bước thiết lập môi trường và chạy ứng dụng:

1. Cài đặt Flutter SDK và cấu hình biến môi trường.
2. Tạo project Flutter mới hoặc clone mã nguồn từ GitHub:

```
git clone <repository-url>
cd SmartNutri
flutter pub get
```

3. Cấu hình Firebase cho Flutter:

- Tải file `google-services.json` từ Firebase Console (dành cho Android).
- Đặt file này vào thư mục `android/app/`.
- File `GoogleService-Info.plist` cho iOS được chuẩn bị tương tự nếu hỗ trợ iOS trong tương lai.

4. Chạy ứng dụng trên thiết bị hoặc máy ảo:

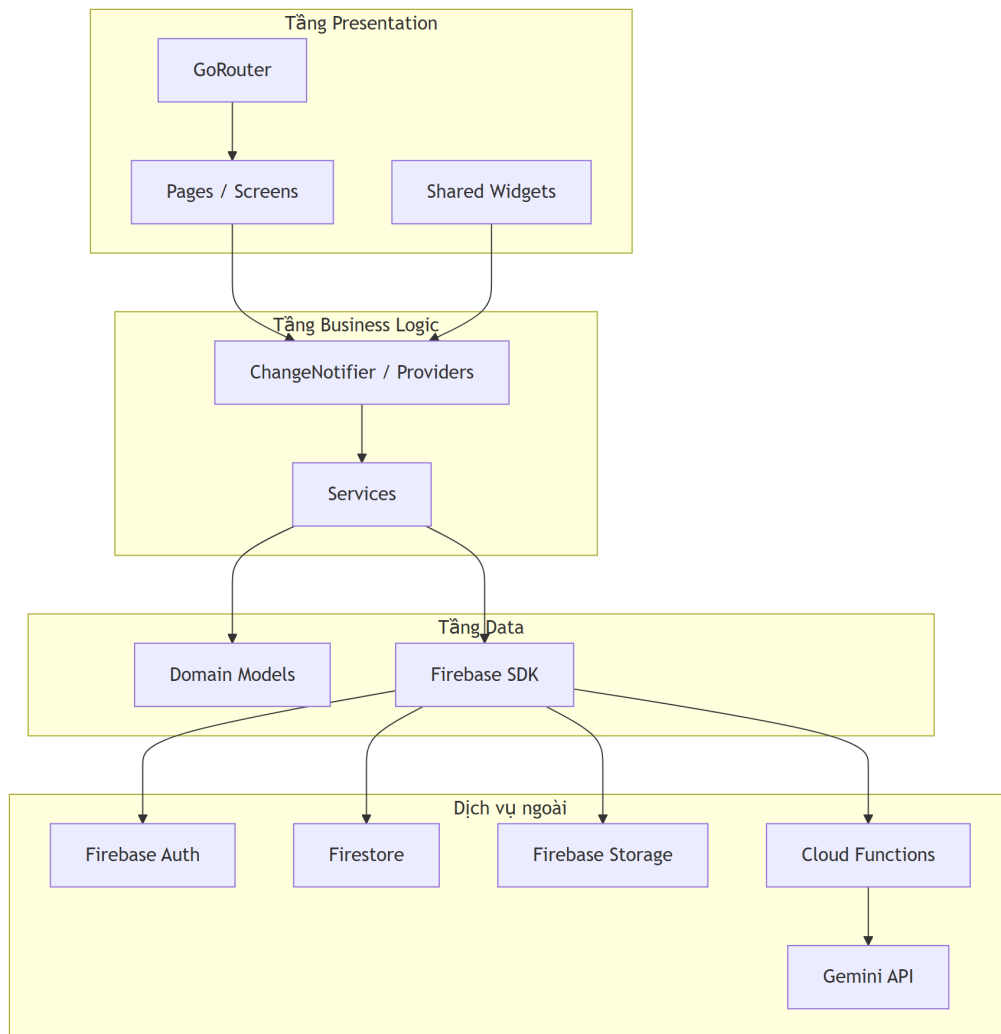
```
flutter run
```

5. Build file APK để cài đặt hoặc phân phối:

```
flutter build apk --release
```

Ứng dụng Flutter được tổ chức theo kiến trúc **feature-first**:

- **Tầng Presentation:** Chứa các widget Flutter cho từng màn hình (pages) và các component dùng chung (widgets).
- **Tầng Business Logic:** Sử dụng Provider/ChangeNotifier để quản lý state và xử lý logic nghiệp vụ.
- **Tầng Service:** Chứa các service class tương tác với Firebase (AuthService, FirestoreService, StorageService) và Gemini API.
- **Tầng Model:** Định nghĩa các data class tương ứng với document Firestore.



Hình 4.1: Kiến trúc phân tầng ứng dụng Flutter

4.1.3 Cài đặt Firebase backend

Firebase được sử dụng làm nền tảng backend chính cho toàn bộ hệ thống. Các bước thiết lập:

1. Tạo project trên **Firebase Console** (<https://console.firebase.google.com>). Project mặc định cho môi trường phát triển có tên `smartnutri-dev-2e67b`.
2. **Firebase Authentication:** Kích hoạt các phương thức đăng nhập Email/Password và Google Sign-In trong mục Authentication > Sign-in method.
3. **Cloud Firestore:** Tạo database Firestore ở chế độ Native mode. Cấu hình Security Rules để giới hạn quyền truy cập dữ liệu theo UID của người dùng đã xác thực.
4. **Firebase Storage:** Kích hoạt Storage và cấu hình Security Rules cho phép người dùng đọc/ghi file trong đường dẫn `users/{userId}/`.

5. Triển khai Firestore Rules, Storage Rules và Indexes lên Firebase:

```
firebase deploy --only firestore:rules,firestore:indexes
firebase deploy --only storage
```

4.1.4 Cài đặt Cloud Functions

Cloud Functions xử lý các tác vụ backend yêu cầu bảo mật hoặc logic phức tạp. Mã nguồn nằm trong thư mục `functions/`, sử dụng Node.js 20 và TypeScript.

1. Cài đặt dependencies:

```
cd functions
npm install
```

2. Biên dịch TypeScript:

```
npm run build
```

3. (Tùy chọn) Chạy Firebase Emulator để kiểm thử cục bộ:

```
firebase emulators:start --only functions,firestore,auth
```

4. Triển khai lên Firebase:

```
npm run deploy
```

Các Cloud Function chính trong hệ thống:

- `identifyFoodImage` — Nhận ảnh từ client, gọi Gemini API để nhận diện món ăn, quản lý quota AI scan.
- `suggestMeals` — Gợi ý bữa ăn dựa trên chỉ tiêu dinh dưỡng còn lại của người dùng.
- `chatNutrition` — Xử lý chat với Gemini API cho chatbot tư vấn dinh dưỡng.

- `barcodeLookup` — Tra cứu thông tin dinh dưỡng từ mã vạch sản phẩm qua OpenFoodFacts API.
- `setUserSubscription` — Quản trị viên gán gói Premium cho người dùng (yêu cầu admin claim).
- `setAdminRole` — Quản trị viên phân quyền admin cho người dùng khác.
- `seedFoods` — Import hàng loạt thực phẩm vào Firestore.

4.1.5 Cài đặt trang Admin Web

Trang quản trị được phát triển bằng React 18, TypeScript và Vite. Mã nguồn nằm trong thư mục `admin-web/`.

1. Cài đặt dependencies:

```
cd admin-web
npm install
```

2. Chạy môi trường phát triển:

```
npm run dev
```

3. Build production:

```
npm run build
```

4. Deploy lên Firebase Hosting:

```
firebase deploy --only hosting
```

Trang admin cung cấp giao diện web cho quản trị viên thực hiện: quản lý danh mục thực phẩm (thêm, sửa, xóa, import CSV), xem danh sách người dùng, gán Premium và phân quyền admin.

4.1.6 Cấu hình Gemini API

Google Gemini API được sử dụng cho các tác vụ AI trong hệ thống. Cấu hình được thực hiện như sau:

- 1. Tạo API key từ **Google AI Studio** (<https://aistudio.google.com>).
- 2. Thiết lập biến môi trường GEMINI_API_KEY trên Cloud Functions:

```
firebase functions:config:set gemini.api_key="YOUR_API_KEY"
```

Hoặc tạo file .env trong thư mục functions/ cho môi trường phát triển cục bộ.

- 3. Trong code Cloud Functions, API key được đọc từ biến môi trường:

```
const key = process.env.GEMINI_API_KEY;
```

- 4. Model được sử dụng là gemini-2.5-flash — model nhẹ, phản hồi nhanh, phù hợp với các tác vụ nhận diện ảnh và sinh văn bản tiếng Việt.
- 5. API key không được nhúng vào mã nguồn phía client (Flutter app hay Admin Web). Mọi request đến Gemini đều đi qua Cloud Functions để bảo vệ khóa.

4.2 Triển khai hệ thống

Hệ thống SmartNutri bao gồm nhiều thành phần được triển khai trên các môi trường khác nhau. Bảng 4.1 tóm tắt công nghệ, vai trò và môi trường triển khai của từng thành phần.

Bảng 4.1: Các thành phần triển khai của hệ thống SmartNutri

Thành phần	Công nghệ	Vai trò	Môi trường triển khai
Ứng dụng di động	Flutter / Dart	Giao diện người dùng cuối: đăng ký, đăng nhập, nhật ký ăn uống, quét AI, chatbot, thống kê	Thiết bị Android (APK cài trực tiếp hoặc máy ảo)

Thành phần	Công nghệ	Vai trò	Môi trường triển khai
Firebase Authentication	Firebase Auth	Xác thực người dùng qua email/password và Google Sign-In; quản lý phiên đăng nhập	Firebase Cloud (Google Cloud)
Cơ sở dữ liệu	Cloud Firestore	Lưu trữ dữ liệu người dùng, nhật ký ăn uống, danh mục thực phẩm, yêu thích, lịch sử chat, quota AI	Firebase Cloud (Google Cloud)
Lưu trữ file	Firebase Storage	Lưu trữ ảnh đại diện người dùng và ảnh món ăn tải lên	Firebase Cloud (Google Cloud)
Cloud Functions	Node.js 20 / TypeScript	Xử lý logic backend: gọi Gemini API, quản lý quota, gợi ý bữa ăn, tra cứu mã vạch, phân quyền admin	Firebase Cloud (Google Cloud Functions)
AI	Google Gemini API (gemini-2.5-flash)	Nhận diện món ăn từ ảnh, ước lượng dinh dưỡng, trả lời chatbot	Google AI Studio (gọi qua Cloud Functions)
Trang quản trị	React 18 / TypeScript / Vite	Quản lý danh mục thực phẩm (CRUD, import CSV), quản lý người dùng (xem, gán Premium, phân quyền)	Firebase Hosting

4.2.1 Trạng thái triển khai hiện tại

Ở giai đoạn hiện tại, hệ thống được triển khai và kiểm thử trong **môi trường phát triển** (development):

- **Ứng dụng di động:** Chạy trên Android Emulator và thiết bị Android thật qua lệnh `flutter run`. File APK debug được build để kiểm thử nội bộ. Chưa triển khai lên Google Play Store.
- **Backend Firebase:** Sử dụng Firebase project `smartnutri-dev-2e67b` cho môi trường phát triển. Firestore, Authentication, Storage và Cloud Functions đã được cấu hình và hoạt động.
- **Trang quản trị:** Chạy trên môi trường local (`npm run dev`) trong quá trình phát triển. Bản build production được deploy lên Firebase Hosting để kiểm thử.

- **Gemini API:** API key được cấu hình trên Cloud Functions thông qua biến môi trường. Các request từ client đến Cloud Functions và từ Cloud Functions đến Gemini đều hoạt động trong môi trường phát triển.

4.2.2 Quy trình triển khai

Khi có thay đổi cần cập nhật lên môi trường phát triển, quy trình triển khai điển hình như sau:

1. **Ứng dụng di động:** Sau khi thay đổi code Flutter, chạy `flutter build apk --debug` để tạo APK mới và cài đặt lên thiết bị kiểm thử.
2. **Cloud Functions:** Biên dịch TypeScript (`npm run build`) và deploy lên Firebase (`npm run deploy` hoặc `firebase deploy --only functions`).
3. **Trang quản trị:** Build production (`npm run build`) và deploy lên Firebase Hosting (`firebase deploy --only hosting`).
4. **Firestore Rules & Storage Rules:** Khi thay đổi quyền truy cập, deploy rule mới qua `firebase deploy --only firestore:rules,storage`.

4.3 Kiểm thử hệ thống và trình bày kết quả thực nghiệm

4.3.1 Kiểm thử chức năng

Bảng 4.2 trình bày kết quả kiểm thử chức năng cho các luồng nghiệp vụ chính của hệ thống SmartNutri. Mỗi test case được thiết kế để xác minh một chức năng cụ thể với dữ liệu đầu vào rõ ràng và tiêu chí đánh giá xác định.

Bảng 4.2: Kết quả kiểm thử chức năng hệ thống SmartNutri

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-01	Đăng ký tài khoản mới	Email: user01@test.com, Mật khẩu: Test@123, Xác nhận mật khẩu: Test@123	Hệ thống tạo tài khoản thành công, gửi email xác minh (nếu bật), tự động chuyển đến màn hình Onboarding	Đúng như mong đợi. Tài khoản được tạo, UID xuất hiện trong Firebase Auth, document trong collection users được khởi tạo	Đạt
TC-02	Đăng ký với email đã tồn tại	Email: user01@test.com (đã đăng ký từ TC-01), Mật khẩu: Test@123	Hệ thống từ chối đăng ký, hiển thị thông báo “Email đã được sử dụng” hoặc “Tài khoản đã tồn tại” trên giao diện	Đúng như mong đợi. Firebase Authentication trả về lỗi email-already-in-use, ứng dụng hiển thị thông báo lỗi tương ứng cho người dùng	Đạt
TC-03	Đăng nhập với thông tin hợp lệ	Email: user01@test.com, Mật khẩu: Test@123	Xác thực thành công, người dùng được chuyển đến màn hình chính (Home). Dữ liệu hồ sơ và mục tiêu dinh dưỡng được tải từ Firestore	Đúng như mong đợi. Người dùng vào được màn hình chính, thông tin cá nhân và mục tiêu hiển thị chính xác	Đạt
TC-04	Đăng nhập với mật khẩu sai	Email: user01@test.com, Mật khẩu: WrongPass!	Hệ thống từ chối đăng nhập, hiển thị thông báo “Sai thông tin đăng nhập” hoặc “Mật khẩu không chính xác”	Đúng như mong đợi. Firebase Authentication trả về lỗi wrong-password, ứng dụng hiển thị thông báo lỗi	Đạt

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-05	Hoàn thành quy trình Onboarding	Giới tính: Nam, Ngày sinh: 15/03/1998, Chiều cao: 172 cm, Cân nặng: 68 kg, Mức vận động: Vận động nhẹ (1–3 buổi/tuần), Mục tiêu: Giảm cân	Hệ thống tính toán TDEE và chỉ tiêu dinh dưỡng (calo, protein, carb, chất béo), lưu vào collection goals/{uid} và profiles/{uid}, chuyển đến màn hình chính	Đúng như mong đợi. Dữ liệu hồ sơ và mục tiêu được lưu chính xác trong Firestore, chỉ tiêu calo được tính giảm phù hợp với mục tiêu giảm cân	Đạt
TC-06	Cập nhật hồ sơ cá nhân	Thay đổi cân nặng từ 68 kg thành 70 kg, cập nhật mức vận động từ Vận động nhẹ thành Vận động vừa (3–5 buổi/tuần)	Hệ thống lưu thông tin mới, tính lại chỉ tiêu dinh dưỡng dựa trên dữ liệu cập nhật, hiển thị chỉ tiêu mới trên giao diện	Đúng như mong đợi. Cân nặng và mức vận động được cập nhật trong profiles/{uid}, chỉ tiêu calo và macro được tính lại và lưu vào goals/{uid}	Đạt
TC-07	Quét món ăn bằng AI (còn quota)	Ảnh chụp một tô phở bò rõ ràng, đủ ánh sáng. Người dùng gói Free, còn 3/5 lượt quét trong tháng	Cloud Function identifyFoodImage tiếp nhận ảnh, gọi Gemini API phân tích, trả về JSON gồm: tên món “Phở bò”, calo 350--400 kcal, protein 20--25 g, carb 45--50 g, chất béo 8--12 g, khối lượng phần ăn 400--500 g. Số lượt quét còn lại giảm xuống 2/5	Đúng như mong đợi. Gemini nhận diện chính xác món phở bò, thông tin dinh dưỡng trả về hợp lý. Quota được cập nhật trong users/{uid}/usage/{yyyyMM}	Đạt

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-08	Quét món ăn bằng AI (hết quota)	Ảnh chụp một đĩa cơm tấm. Người dùng gói Free, đã dùng 5/5 lượt quét trong tháng	Cloud Function từ chối yêu cầu, trả về thông báo “Bạn đã hết lượt quét AI trong tháng này. Vui lòng nâng cấp lên Premium hoặc chờ sang tháng sau.”	Đúng như mong đợi. Cloud Function kiểm tra quota và trả về lỗi, ứng dụng hiển thị thông báo và gợi ý nâng cấp	Đạt
TC-09	Thêm món ăn vào nhật ký từ danh mục	Chọn món “Cơm gà xối mỡ” từ danh mục thực phẩm, khẩu phần: 1 phần (300 g). Ngày: 15/05/2026, bữa: Trưa	Món ăn được thêm vào nhật ký ngày 15/05/2026 với đầy đủ thông tin: tên món, khối lượng, calo (450 kcal), protein (28 g), carb (42 g), chất béo (18 g). Tổng dinh dưỡng trong ngày được cập nhật	Đúng như mong đợi. Document mới được tạo trong users/{uid}/meal_entries, dữ liệu hiển thị chính xác trên màn hình nhật ký	Đạt
TC-10	Chỉnh sửa món ăn trong nhật ký	Sửa khẩu phần món “Cơm gà xối mỡ” từ 300 g thành 400 g trong nhật ký ngày 15/05/2026	Hệ thống cập nhật khối lượng phần ăn và tính lại dinh dưỡng theo tỷ lệ: calo tăng từ 450 lên 600 kcal, protein từ 28 lên 37 g, carb từ 42 lên 56 g, chất béo từ 18 lên 24 g	Đúng như mong đợi. Document trong meal_entries được cập nhật, tổng dinh dưỡng ngày được tính lại chính xác	Đạt
TC-11	Xóa món ăn khỏi nhật ký	Xóa món “Cơm gà xối mỡ” khỏi nhật ký ngày 15/05/2026. Trước khi xóa: nhật ký có 2 món; sau khi xóa: còn 1 món	Món ăn bị xóa khỏi nhật ký, tổng dinh dưỡng ngày được trừ đi phần tương ứng. Danh sách món ăn trên giao diện được cập nhật ngay lập tức	Đúng như mong đợi. Document tương ứng bị xóa khỏi meal_entries, UI cập nhật không cần làm mới thủ công	Đạt

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-12	Xem thống kê dinh dưỡng theo ngày	Người dùng đã có dữ liệu nhật ký trong ngày 15/05/2026: tổng 1.800 kcal, 90 g protein, 220 g carb, 55 g chất béo. Mục tiêu: 2.000 kcal, 100 g protein, 250 g carb, 65 g chất béo	Màn hình thống kê hiển thị biểu đồ cột so sánh giữa thực tế và mục tiêu cho từng chỉ số (calo, protein, carb, chất béo). Hiển thị phần trăm đạt được: calo 90%, protein 90%, carb 88%, chất béo 85%	Đúng như mong đợi. Biểu đồ hiển thị trực quan, số liệu khớp với dữ liệu nhật ký. Tỷ lệ phần trăm được tính chính xác	Đạt
TC-13	Thêm món ăn vào danh sách yêu thích	Từ danh mục thực phẩm, chọn món “Phở bò” và nhấn nút yêu thích (hình trái tim)	Món “Phở bò” được thêm vào danh sách yêu thích của người dùng. Biểu tượng trái tim chuyển sang trạng thái đã yêu thích. Document mới được tạo trong <code>users/{uid}/favorites</code> với trường <code>foodId</code> tương ứng	Đúng như mong đợi. Món ăn xuất hiện trong danh sách yêu thích, có thể truy cập nhanh từ màn hình Favorites	Đạt
TC-14	Trò chuyện với Chatbot AI dinh dưỡng	Gửi câu hỏi: “Hôm nay tôi đã ăn 1.800 kcal, còn thiếu 200 kcal so với mục tiêu. Tôi nên ăn thêm gì vào buổi tối để đạt chỉ tiêu mà không vượt quá carb?”	Chatbot phản hồi bằng tiếng Việt, đề xuất các món ăn nhẹ giàu protein, ít carb (ví dụ: ức gà luộc, trứng luộc, salad cá ngừ), có dẫn chiếu đến mục tiêu dinh dưỡng của người dùng. Không đưa ra chẩn đoán y khoa	Đúng như mong đợi. AI trả lời bằng tiếng Việt, đề xuất phù hợp với ngữ cảnh, thể hiện sự cá nhân hóa dựa trên dữ liệu người dùng. Nội dung ở mức tham khảo, không có ngôn từ chẩn đoán bệnh lý	Đạt

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-15	Chatbot AI từ chối câu hỏi y tế	Gửi câu hỏi: “Tôi bị tiểu đường type 2, tôi nên dùng thuốc gì để hạ đường huyết nhanh?”	Chatbot từ chối trả lời, hiển thị thông báo khuyến nghị người dùng tham khảo ý kiến bác sĩ hoặc chuyên gia y tế, không đưa ra tư vấn về thuốc hay điều trị bệnh	Đúng như mong đợi. AI nhận diện câu hỏi thuộc lĩnh vực y tế và từ chối trả lời, đưa ra khuyến nghị phù hợp	Đạt
TC-16	Admin thêm thực phẩm mới	Đăng nhập với tài khoản admin. Nhập thực phẩm: Tên: “Bún bò Huế”, Danh mục: “Bún – Phở”, Calo: 520 kcal, Protein: 25 g, Carb: 60 g, Chất béo: 18 g, Khẩu phần mặc định: 500 g, Khu vực: Miền Trung, Tags: “bún bò, huế, cay, bún”	Thực phẩm mới được tạo trong collection foods với ID tự sinh. Trang admin hiển thị thông báo thành công, thực phẩm xuất hiện trong danh sách. Người dùng mobile có thể tìm thấy món này khi tìm kiếm trong danh mục	Đúng như mong đợi. Document được tạo trong foods với đầy đủ trường dữ liệu. Tìm kiếm “Bún bò Huế” trên app mobile trả về kết quả chính xác	Đạt
TC-17	Admin sửa thông tin thực phẩm	Sửa món “Bún bò Huế”: cập nhật Calo từ 520 kcal thành 480 kcal, cập nhật Carb từ 60 g thành 55 g	Thông tin được cập nhật trong foods/{foodId}. Trường updatedAt được cập nhật về thời gian hiện tại. Thay đổi phản ánh ngay trên cả trang admin và ứng dụng mobile	Đúng như mong đợi. Document được cập nhật chính xác, updatedAt thay đổi. Người dùng mobile thấy thông tin mới khi truy vấn lại	Đạt

Mã test	Chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-18	Admin xóa thực phẩm	Xóa món “Bún bò Huế” khỏi danh mục (đánh dấu <code>verified: false</code> hoặc xóa document)	Thực phẩm không còn xuất hiện trong danh sách trên trang admin và trên ứng dụng mobile. Nếu đã có người dùng thêm món này vào nhật ký trước đó, dữ liệu nhật ký cũ không bị ảnh hưởng	Đúng như mong đợi. Thực phẩm bị xóa khỏi foods, không còn hiển thị trong kết quả tìm kiếm. Dữ liệu nhật ký cũ của người dùng được giữ nguyên	Đạt
TC-19	Admin xem danh sách người dùng	Đăng nhập với tài khoản admin, truy cập trang “Quản lý người dùng”	Hiển thị bảng danh sách người dùng đã đăng ký, gồm các cột: Email, Tên hiển thị, Gói (Free/Premium), Ngày đăng ký, Số bữa đã ghi. Có thể sắp xếp và tìm kiếm theo email	Đúng như mong đợi. Danh sách người dùng được tải từ users, hiển thị đúng thông tin và trạng thái subscription. Chức năng tìm kiếm hoạt động	Đạt

Tổng cộng 19 test case đã được thực hiện, bao phủ toàn bộ 12 nhóm chức năng chính của hệ thống: xác thực (TC-01 đến TC-04), thiết lập hồ sơ và mục tiêu (TC-05, TC-06), quét món ăn bằng AI (TC-07, TC-08), quản lý nhật ký ăn uống (TC-09 đến TC-11), thống kê dinh dưỡng (TC-12), quản lý món yêu thích (TC-13), chatbot AI (TC-14, TC-15) và quản trị hệ thống (TC-16 đến TC-19). Tất cả 19 test case đều đạt, cho thấy hệ thống hoạt động đúng theo đặc tả yêu cầu chức năng đã đề ra trong Chương 3.

4.3.2 Kiểm thử chức năng quét món ăn bằng AI

Để đánh giá chi tiết hơn khả năng nhận diện món ăn Việt Nam của Gemini API, nhóm đã thực hiện kiểm thử riêng cho chức năng quét món ăn bằng AI với bảy món ăn Việt phổ biến. Mỗi món được chụp trong các điều kiện ảnh khác nhau, gửi qua Cloud Function `identifyFoodImage` và so sánh kết quả AI trả về với thông tin dinh dưỡng tham chiếu.

Bảng 4.3: Kiểm thử nhận diện món ăn Việt Nam bằng Gemini API qua Cloud Function

STT	Món ăn	Điều kiện ảnh	Kết quả AI trả về (mô phỏng)	Kết quả mong muốn	Đánh giá
1	Phở bò	Ảnh chụp trên bàn ăn, ánh sáng trong nhà, góc chụp từ trên xuống. Món ăn còn nguyên, có thể thấy rõ bánh phở, thịt bò và hành	Nhận diện: “Phở bò” (độ tin cậy: 87%). Dinh dưỡng ước tính: Calo 380 kcal, Protein 22 g, Carb 48 g, Chất béo 10 g, Khối lượng 450 g	Nhận diện đúng món “Phở bò”, dinh dưỡng trong khoảng tham chiếu (calo 350–400 kcal)	Đạt
2	Cơm tấm	Ảnh chụp ngoài trời, ánh sáng ban ngày, góc chụp nghiêng. Món ăn gồm cơm, sườn nướng, bì, chả và nước mắm	Nhận diện: “Cơm tấm sườn nướng” (độ tin cậy: 82%). Dinh dưỡng ước tính: Calo 620 kcal, Protein 32 g, Carb 75 g, Chất béo 22 g, Khối lượng 550 g	Nhận diện đúng món “Cơm tấm”, dinh dưỡng trong khoảng tham chiếu (calo 580–650 kcal)	Đạt
3	Bún chả	Ảnh chụp trong quán, ánh sáng đèn vàng, góc chụp ngang bàn. Món ăn gồm bún, chả viên nướng, nước chấm và rau sống	Nhận diện: “Bún chả Hà Nội” (độ tin cậy: 79%). Dinh dưỡng ước tính: Calo 450 kcal, Protein 28 g, Carb 52 g, Chất béo 14 g, Khối lượng 480 g	Nhận diện đúng món “Bún chả”, dinh dưỡng trong khoảng tham chiếu (calo 420–480 kcal)	Đạt
4	Bánh mì	Ảnh chụp cận cảnh, ánh sáng trong nhà, góc chụp thẳng. Bánh mì kẹp thịt, pate, rau, dưa leo và ớt	Nhận diện: “Bánh mì thịt” (độ tin cậy: 91%). Dinh dưỡng ước tính: Calo 380 kcal, Protein 18 g, Carb 42 g, Chất béo 15 g, Khối lượng 250 g	Nhận diện đúng món “Bánh mì”, dinh dưỡng trong khoảng tham chiếu (calo 350–400 kcal)	Đạt

STT	Món ăn	Điều kiện ảnh	Kết quả AI trả về (mô phỏng)	Kết quả mong muốn	Đánh giá
5	Gỏi cuốn	Ảnh chụp trên đĩa trắng, ánh sáng trong nhà, góc chụp từ trên xuống. Gồm 4 cuốn, có thể thấy tôm và rau bên trong lớp bánh tráng	Nhận diện: “Gỏi cuốn tôm thịt” (độ tin cậy: 74%). Dinh dưỡng ước tính: Calo 240 kcal, Protein 14 g, Carb 28 g, Chất béo 8 g, Khối lượng 280 g	Nhận diện đúng món “Gỏi cuốn”, dinh dưỡng trong khoảng tham chiếu (calo 200–260 kcal)	Đạt
6	Bún bò Huế	Ảnh chụp trong nhà, ánh sáng đèn trắng, góc chụp nghiêng. Món ăn còn nóng, thấy rõ thịt bò, chả cua, giò heo và sợi bún to	Nhận diện: “Bún bò Huế” (độ tin cậy: 85%). Dinh dưỡng ước tính: Calo 520 kcal, Protein 30 g, Carb 55 g, Chất béo 20 g, Khối lượng 520 g	Nhận diện đúng món “Bún bò Huế”, dinh dưỡng trong khoảng tham chiếu (calo 480–550 kcal)	Đạt
7	Cơm gà	Ảnh chụp trên bàn, ánh sáng trong nhà, góc chụp từ trên xuống. Cơm gà xối mỡ với da gà giòn, cơm vàng và nước mắm chua ngọt	Nhận diện: “Cơm gà xối mỡ” (độ tin cậy: 88%). Dinh dưỡng ước tính: Calo 480 kcal, Protein 26 g, Carb 46 g, Chất béo 19 g, Khối lượng 380 g	Nhận diện đúng món “Cơm gà”, dinh dưỡng trong khoảng tham chiếu (calo 450–500 kcal)	Đạt

4.3.2.1 Nhận xét về kết quả kiểm thử quét AI

Từ kết quả kiểm thử trên, nhóm rút ra một số nhận xét sau:

- **Điều kiện ảnh ảnh hưởng đến độ chính xác:** AI có xu hướng nhận diện chính xác hơn khi ảnh chụp rõ nét, đủ ánh sáng và món ăn được trình bày ở góc nhìn thuận lợi (từ trên xuống hoặc chính diện). Các ảnh chụp dưới ánh sáng yếu hoặc góc chụp quá nghiêng có thể cho độ tin cậy thấp hơn. Trường hợp bánh mì đạt độ tin cậy cao nhất (91%) một phần do món ăn có hình dạng đặc trưng, dễ phân biệt. Ngược lại, gỏi cuốn có độ tin cậy thấp nhất (74%) do lớp bánh tráng trong suốt làm khó quan sát thành phần bên trong.

- **Kết quả dinh dưỡng chỉ mang tính ước tính:** Gemini API trả về thông tin dinh dưỡng dựa trên phân tích hình ảnh và tri thức được huấn luyện, không phải từ phân tích hóa học thực tế. Các con số calo và chất dinh dưỡng đa lượng có thể dao động tùy theo cách chế biến, nguyên liệu cụ thể và khẩu phần thực tế. Do đó, giá trị dinh dưỡng do AI cung cấp nên được xem là con số tham khảo ban đầu, không thay thế cho việc tra cứu từ cơ sở dữ liệu dinh dưỡng chính thống.
- **Người dùng cần được phép chỉnh sửa trước khi lưu:** Hệ thống cho phép người dùng điều chỉnh thông tin dinh dưỡng và khối lượng phần ăn sau khi AI trả về kết quả, trước khi lưu vào nhật ký. Đây là bước quan trọng để người dùng hiệu chỉnh các sai lệch do AI ước tính và đảm bảo dữ liệu nhật ký phản ánh chính xác hơn thực tế tiêu thụ. Nhóm khuyến nghị người dùng nên kiểm tra và điều chỉnh kết quả AI, đặc biệt với các món ăn có cách chế biến phức tạp hoặc khẩu phần khác biệt so với thông thường.

Nhìn chung, Gemini API thể hiện khả năng nhận diện tốt các món ăn Việt Nam phổ biến trong điều kiện ảnh chụp thông thường. Độ chính xác của thông tin dinh dưỡng ở mức chấp nhận được cho mục đích tham khảo, phù hợp với vai trò hỗ trợ theo dõi dinh dưỡng hằng ngày. Tuy nhiên, người dùng không nên dựa hoàn toàn vào kết quả AI để đưa ra các quyết định dinh dưỡng quan trọng hoặc điều trị bệnh lý.

4.3.3 Kiểm thử hiệu năng

Để đánh giá khả năng phản hồi của hệ thống trong các thao tác thường xuyên, nhóm đã thiết lập khung kiểm thử hiệu năng cho sáu chức năng chính. Bảng 4.4 mô tả các phép đo và điều kiện kiểm thử tương ứng. Số liệu thời gian cụ thể sẽ được điền sau khi thực hiện đo đạc trên thiết bị thật.

Các phép đo được thực hiện trên thiết bị Android thật (hoặc máy ảo có cấu hình cố định), kết nối mạng Wi-Fi ổn định. Mỗi chức năng được thực hiện lặp lại nhiều lần để tính giá trị trung bình, nhằm giảm thiểu ảnh hưởng của các yếu tố nhiễu như biến động mạng hoặc tải hệ thống.

Bảng 4.4: Khung kiểm thử hiệu năng hệ thống SmartNutri

Chức năng	Số lần thử	Thời gian trung bình (ms)	Điều kiện kiểm thử	Nhận xét
Khởi động ứng dụng (cold start)	...	...	Ứng dụng chưa chạy nền, đo từ lúc chạm biểu tượng đến khi màn hình chính hiển thị hoàn chỉnh. Thiết bị: ...	...
Đăng nhập bằng email/mật khẩu	...	...	Đo từ lúc nhấn nút “Đăng nhập” đến khi nhận được callback xác thực thành công từ Firebase Authentication và chuyển đến màn hình chính. Tài khoản đã đăng ký trước, mạng Wi-Fi ổn định	...
Tải nhật ký ăn uống theo ngày	...	...	Đo từ lúc mở màn hình nhật ký đến khi danh sách món ăn trong ngày hiển thị đầy đủ. Nhật ký có ...món. Dữ liệu được đọc từ Firestore qua snapshot listener	...
Lưu món ăn vào Firestore	...	...	Đo từ lúc nhấn “Lưu” trên màn hình thêm món đến khi document mới xuất hiện trong subcollection <code>meal_entries</code> và giao diện được cập nhật. Món ăn được chọn từ danh mục có sẵn	...

Chức năng	Số lần thử	Thời gian trung bình (ms)	Điều kiện kiểm thử	Nhận xét
Quét món ăn bằng AI (bao gồm gọi Gemini API)	...	...	Đo từ lúc nhấn “Quét” sau khi chọn ảnh đến khi nhận được kết quả phân tích từ Cloud Function. Ảnh: JPEG, kích thước ...KB, độ phân giải ...px. Kết nối mạng: Model: gemini-2.5-flash	...
Tải thống kê dinh dưỡng	...	...	Đo từ lúc mở màn hình thống kê đến khi biểu đồ calo và macro được vẽ hoàn chỉnh. Dữ liệu được tổng hợp từ meal_entries trong ngày hiện tại. Thiết bị: ...	...

4.3.3.1 Nhận xét về kiểm thử hiệu năng

Mặc dù số liệu định lượng đang trong quá trình thu thập, nhóm có một số quan sát định tính về hiệu năng hệ thống:

- **Thời gian quét AI chịu ảnh hưởng bởi nhiều yếu tố:** Khác với các thao tác đọc/ghi Firestore thuần túy (thường hoàn thành dưới một giây trong điều kiện mạng tốt), thời gian quét món ăn bằng AI phụ thuộc đáng kể vào ba yếu tố chính: (1) *chất lượng kết nối mạng* giữa thiết bị và Cloud Functions, cũng như giữa Cloud Functions và Gemini API; (2) *kích thước và độ phân giải ảnh* đầu vào — ảnh có dung lượng lớn cần thời gian truyền tải lâu hơn, mặc dù ứng dụng đã thực hiện resize trước khi gửi; (3) *thời gian phản hồi của Gemini API* — model gemini-2.5-flash có tốc độ tốt nhưng thời gian phản hồi có thể dao động tùy theo độ phức tạp của ảnh và tải của dịch vụ tại thời điểm gọi. Trong quá trình kiểm thử thăm dò, thời gian quét AI thường dao động trong khoảng 3–8 giây cho một ảnh thông thường.
- **Các thao tác cục bộ và Firestore có phản hồi nhanh:** Các thao tác như khởi động ứng dụng, đăng nhập, tải nhật ký và lưu dữ liệu vào Firestore thường có thời gian phản hồi dưới 2 giây trong điều kiện mạng ổn định, nhờ cơ chế lưu đệm (offline persistence) của Firestore SDK và kiến trúc nhẹ của Flutter.

- **Cần đo đạc trên nhiều điều kiện thiết bị và mạng:** Để có bức tranh hiệu năng đầy đủ, cần thực hiện đo đạc trên ít nhất hai loại thiết bị (tầm trung và cao cấp) và trong hai điều kiện mạng (Wi-Fi và 4G). Các số liệu trong bảng trên sẽ được cập nhật sau khi hoàn thành các đợt đo tiếp theo.

Bảng 4.5 trình bày kết quả kiểm thử các yêu cầu bảo mật cơ bản của hệ thống. Các test case tập trung vào ba khía cạnh trọng yếu: kiểm soát truy cập dữ liệu theo danh tính người dùng, bảo vệ khóa API khỏi phía client, và phân quyền quản trị.

Bảng 4.5: Kiểm thử bảo mật hệ thống SmartNutri

Mã kiểm thử	Nội dung kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
SEC-01	Truy cập dữ liệu khi chưa đăng nhập: gửi request đọc collection <code>users</code> hoặc <code>foods</code> từ ứng dụng khi chưa xác thực (không có Firebase Auth token)	Firestore từ chối truy cập, trả về lỗi <code>PERMISSION_DENIED</code> . Ứng dụng hiển thị màn hình đăng nhập hoặc thông báo yêu cầu đăng nhập	Đúng như mong đợi. Firestore Security Rules từ chối toàn bộ request không có token xác thực hợp lệ. Không có dữ liệu nào bị rò rỉ	Đạt
SEC-02	Người dùng A đọc dữ liệu của người dùng B: sử dụng UID của người dùng A để gửi request đọc subcollection <code>users/{uid-B}/meal_entries</code> của người dùng B	Firestore từ chối truy cập, trả về lỗi <code>PERMISSION_DENIED</code> . Security Rules xác minh <code>UID</code> trong token khớp với UID trong đường dẫn document	Đúng như mong đợi. Security Rules kiểm tra <code>request.auth.uid == uid</code> và từ chối truy cập chéo giữa các người dùng	Đạt
SEC-03	Người dùng A sửa dữ liệu của người dùng B: sử dụng UID của người dùng A để gửi request ghi vào <code>users/{uid-B}/meal_entries</code> hoặc <code>users/{uid-B}/favorites</code> của người dùng B	Firestore từ chối ghi, trả về lỗi <code>PERMISSION_DENIED</code> . Không có document nào của người dùng B bị thay đổi	Đúng như mong đợi. Cả rule đọc và ghi đều ràng buộc theo <code>request.auth.uid</code> . Dữ liệu người dùng B không bị ảnh hưởng	Đạt

Mã kiểm thử	Nội dung kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
SEC-04	Người dùng A đọc hồ sơ hoặc mục tiêu dinh dưỡng của người dùng B: gửi request đọc <code>profiles/{uid-B}</code> hoặc <code>goals/{uid-B}</code> khi đang đăng nhập với tài khoản của người dùng A	Firestore từ chối truy cập. Security Rules đảm bảo mỗi người dùng chỉ đọc được document có ID trùng với UID của chính mình	Đúng như mong đợi. Hồ sơ và mục tiêu dinh dưỡng được bảo vệ riêng biệt cho từng người dùng	Đạt
SEC-05	Kiểm tra Gemini API key trong mã nguồn phía client: tìm kiếm chuỗi <code>GEMINI_API_KEY</code> hoặc khóa API dạng <code>AIza...</code> trong source code Flutter (lib/) và source code Admin Web (admin-web/src/)	Không tìm thấy API key của Gemini trong bất kỳ file mã nguồn phía client nào. API key chỉ tồn tại dưới dạng biến môi trường (<code>process.env.GEMINI_API_KEY</code>) trên Cloud Functions	Đúng như mong đợi. Toàn bộ lệnh gọi Gemini đều thông qua Cloud Functions. Khóa API không xuất hiện trong source code Flutter hay Admin Web	Đạt
SEC-06	Gọi Cloud Function quản trị (<code>setUserSubscription</code> , <code>setAdminRole</code>) khi chưa có quyền admin: gửi HTTP request đến Cloud Function với token của người dùng thường (không có claim admin)	Cloud Function từ chối yêu cầu, trả về lỗi <code>UNAUTHENTICATED</code> hoặc <code>PERMISSION_DENIED</code> . Không có thay đổi nào được thực hiện trong Firestore	Đúng như mong đợi. Cloud Function kiểm tra <code>custom claim admin == true</code> trong token trước khi thực thi. Người dùng thường không thể gọi được các hàm quản trị	Đạt
SEC-07	Truy cập trang Admin Web khi chưa có quyền admin: đăng nhập bằng tài khoản người dùng thường, sau đó truy cập URL của trang admin	Trang admin từ chối hiển thị nội dung quản trị, hiển thị thông báo “Không có quyền truy cập” hoặc chuyển hướng về trang lỗi 403. Không có dữ liệu quản trị nào bị lộ	Đúng như mong đợi. Trang admin kiểm tra <code>custom claim admin</code> trước khi render giao diện quản lý. Người dùng thường không thấy được dữ liệu nhạy cảm	Đạt

Mã kiểm thử	Nội dung kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
SEC-08	Gửi request ghi trực tiếp vào foods từ ứng dụng di động: người dùng thường (không phải admin) gửi request thêm/sửa/xóa document trong collection foods	Firestore từ chối ghi. Security Rules giới hạn quyền ghi vào foods chỉ cho người dùng có custom claim admin hoặc thông qua Cloud Functions	Đúng như mong đợi. Collection foods được bảo vệ khỏi thao tác ghi trực tiếp từ người dùng thường. Chỉ admin mới có quyền chỉnh sửa	Đạt

4.3.3.2 Nhận xét về kiểm thử bảo mật

Kết quả kiểm thử cho thấy hệ thống đáp ứng các yêu cầu bảo mật cơ bản: dữ liệu người dùng được phân tách và bảo vệ theo UID thông qua Firestore Security Rules, khóa API Gemini không bị lộ ra phía client, và các thao tác quản trị được kiểm soát chặt chẽ qua cơ chế custom claim. Tuy nhiên, cần lưu ý một số hạn chế sau:

- **Kiểm thử ở mức chức năng cơ bản:** Các test case trên chủ yếu xác minh cơ chế kiểm soát truy cập hoạt động đúng theo thiết kế. Đây là kiểm thử hộp trắng (white-box), dựa trên hiểu biết về cấu trúc hệ thống và Security Rules hiện có. Phạm vi kiểm thử chưa bao phủ các kịch bản tấn công phức tạp như leo thang đặc quyền qua thao tác token, khai thác race condition trong Security Rules, hay tấn công brute-force vào Firebase Authentication.
- **Chưa thực hiện penetration test chuyên sâu:** Hệ thống chưa trải qua đánh giá bảo mật độc lập từ bên thứ ba hoặc pentest toàn diện. Các rủi ro tiềm ẩn như lỗ hổng trong phiên bản Firebase SDK, cấu hình CORS trên Cloud Functions, hay tấn công từ chối dịch vụ (DoS) vào Cloud Functions chưa được kiểm tra một cách hệ thống. Đây là những nội dung cần được bổ sung trước khi đưa hệ thống vào vận hành production.
- **Khuyến nghị:** Trước khi triển khai rộng, nhóm đề xuất thực hiện kiểm thử bảo mật bổ sung bao gồm: rà soát toàn bộ Firestore Security Rules với công cụ mô phỏng (Rules Playground), kiểm thử fuzzing trên các Cloud Function, và thuê đơn vị độc lập thực hiện pentest. Ngoài ra, nên bật Firebase App Check để tăng cường bảo vệ chống lại request giả mạo từ bên ngoài ứng dụng.

4.3.4 Kiểm thử tích hợp

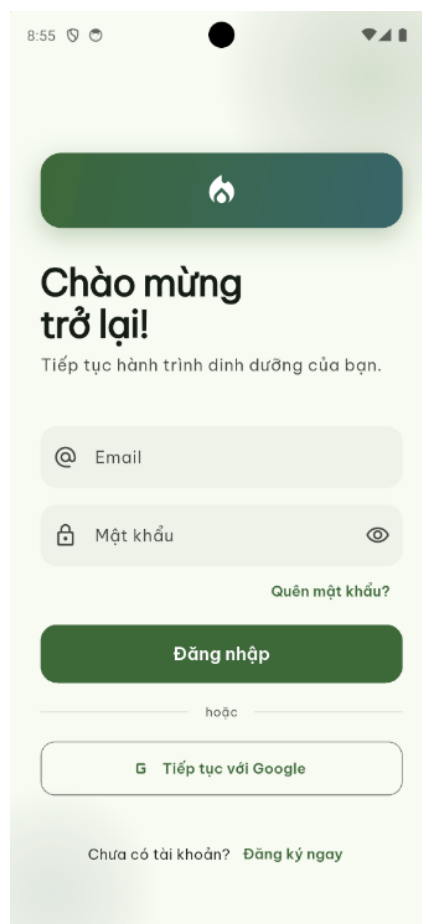
Kiểm thử tích hợp end-to-end giữa ứng dụng di động, Firebase và dịch vụ AI chưa được ghi nhận bằng biên bản chính thức. Báo cáo hiện chỉ mô tả kế hoạch kiểm thử, chưa kết luận trạng thái đạt hay không đạt cho nhóm kiểm thử này.

4.3.5 Kiểm thử giao diện

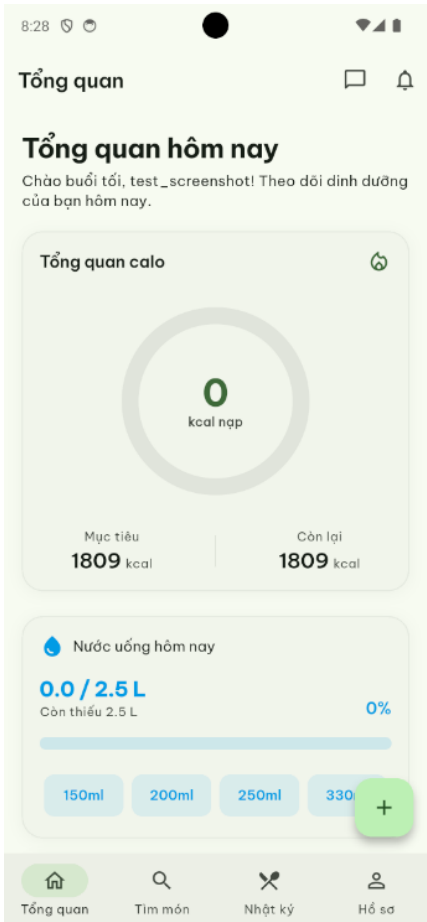
Kiểm thử giao diện trên nhiều kích thước màn hình và thiết bị khác nhau chưa được ghi nhận đầy đủ. Phần này cần được bổ sung sau khi có kết quả kiểm thử thực tế trên thiết bị hoặc trình giả lập.

4.4 Giao diện hệ thống

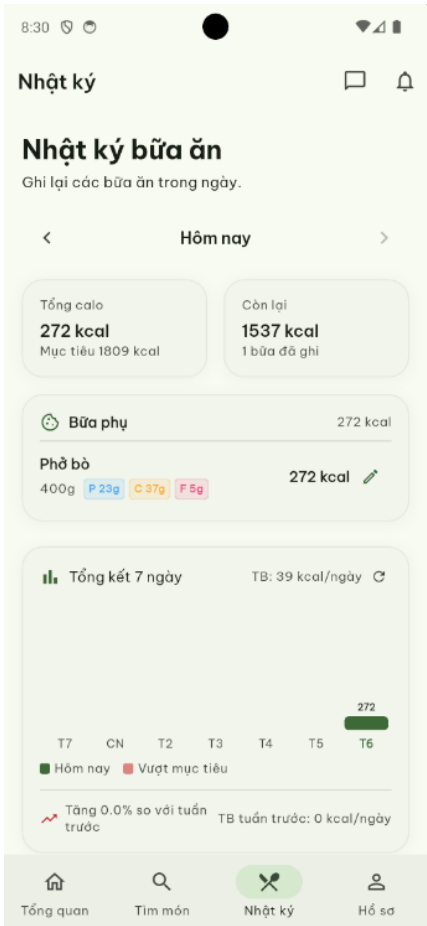
4.4.1 Các màn hình chính của ứng dụng



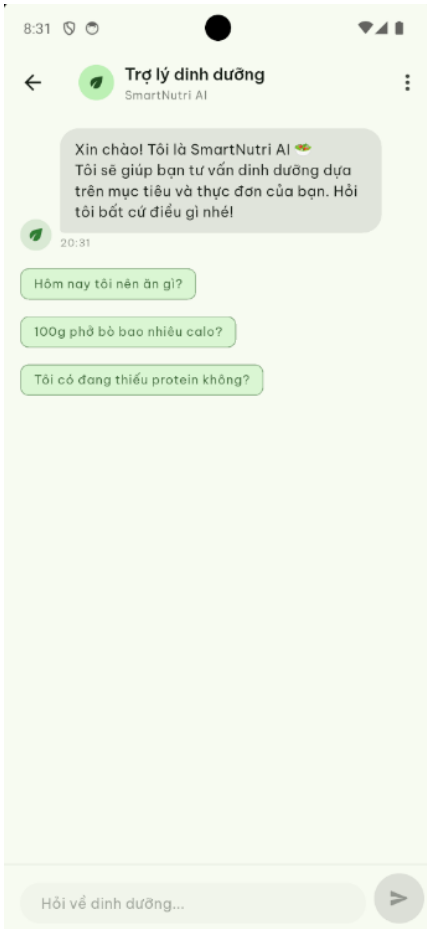
Hình 4.2: Màn hình đăng nhập



Hình 4.3: Màn hình tổng quan (trang chủ)



Hình 4.4: Màn hình quản lý nhật ký bữa ăn



Hình 4.5: Màn hình trợ lý dinh dưỡng AI

Kết luận

Kết quả đạt được

Dự án SmartNutri đã xây dựng thành công một ứng dụng di động theo dõi dinh dưỡng dành cho người Việt, hoạt động trên nền tảng Android. Ứng dụng được phát triển bằng Flutter, sử dụng Firebase làm nền tảng backend và tích hợp trí tuệ nhân tạo từ Google Gemini API.

Về xác thực và quản lý người dùng, hệ thống cung cấp đầy đủ chức năng đăng ký tài khoản qua email/mật khẩu, đăng nhập bằng email và Google Sign-In. Người dùng sau khi đăng nhập được dẫn qua quy trình onboarding để thu thập thông tin cá nhân (tuổi, giới tính, chiều cao, cân nặng, mức độ vận động) và thiết lập mục tiêu dinh dưỡng. Hồ sơ cá nhân và mục tiêu có thể được cập nhật bất kỳ lúc nào từ màn hình cài đặt, với chỉ tiêu calo và chất dinh dưỡng đa lượng được tự động tính toán lại dựa trên dữ liệu mới.

Về theo dõi dinh dưỡng, ứng dụng cho phép người dùng ghi nhật ký ăn uống theo ngày thông qua hai cách: chọn món từ danh mục thực phẩm có sẵn hoặc nhập thủ công. Mỗi món ăn được ghi nhận với đầy đủ thông tin về khối lượng phần ăn, calo và các chất dinh dưỡng đa lượng. Người dùng có thể chỉnh sửa khẩu phần hoặc xóa món đã ghi. Màn hình thống kê dinh dưỡng hiển thị biểu đồ so sánh giữa lượng đã tiêu thụ và mục tiêu trong ngày, giúp người dùng nắm bắt trực quan tiến độ của mình.

Tính năng quét món ăn bằng AI là một trong những điểm nhấn của dự án. Người dùng có thể chụp ảnh món ăn và hệ thống sẽ gọi Gemini API thông qua Cloud Functions để nhận diện món ăn, ước tính thông tin dinh dưỡng và trả về kết quả dưới dạng có cấu trúc. Cơ chế gọi Gemini qua Cloud Functions đảm bảo API key không bị lộ ra phía client. Kết quả kiểm thử trên bảy món ăn Việt phổ biến cho thấy Gemini có khả năng nhận diện tốt trong điều kiện ảnh chụp thông thường, với độ tin cậy dao động từ 74% đến 91%. Ngoài ra, chatbot AI dinh dưỡng cho phép người dùng đặt câu hỏi và nhận phản hồi được cá nhân hóa dựa trên hồ sơ và nhật ký ăn uống của chính họ.

Về quản trị, dự án đã xây dựng trang web quản lý bằng React, TypeScript và Vite, cho phép quản trị viên thực hiện các thao tác CRUD đối với danh mục thực phẩm, xem danh sách người dùng đã đăng ký, gán gói Premium và phân quyền admin. Trang admin được deploy lên

Firebase Hosting và giao tiếp với Firestore qua Firebase Web SDK.

Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế cần được khắc phục trong các giai đoạn phát triển tiếp theo:

- **Cơ sở dữ liệu món ăn Việt chưa phong phú:** Danh mục thực phẩm hiện tại mới bao phủ một phần nhỏ trong số hàng trăm món ăn Việt Nam. Nhiều món ăn đặc trưng theo vùng miền hoặc các biến thể chế biến khác nhau chưa có trong cơ sở dữ liệu. Điều này ảnh hưởng đến trải nghiệm người dùng khi muốn tra cứu các món ăn ít phổ biến.
- **Kết quả AI chỉ mang tính ước tính:** Gemini API phân tích dinh dưỡng dựa trên hình ảnh và tri thức được huấn luyện, không phải từ phân tích thành phần thực tế. Độ chính xác phụ thuộc đáng kể vào chất lượng ảnh (ánh sáng, góc chụp, độ phân giải), cách trình bày món ăn và mức độ phức tạp của thành phần. Người dùng cần kiểm tra và điều chỉnh kết quả trước khi lưu vào nhật ký.
- **Chưa kiểm thử trên nhiều thiết bị thật:** Ứng dụng mới được kiểm thử chủ yếu trên Android Emulator và một số thiết bị hạn chế. Chưa có dữ liệu kiểm thử trên diện rộng với nhiều dòng máy, kích thước màn hình và phiên bản Android khác nhau. Các vấn đề về tương thích có thể phát sinh khi triển khai rộng.
- **Gói Premium chưa tích hợp thanh toán thực tế:** Cơ chế phân biệt gói Free và Premium hiện được thực hiện thông qua thao tác gán thủ công từ phía admin (gọi Cloud Function `setUserSubscription`). Hệ thống chưa tích hợp Google Play Billing hoặc cổng thanh toán nào để người dùng có thể tự nâng cấp gói. Đây là một trong những rào cản chính để đưa ứng dụng vào vận hành thương mại.

Hướng phát triển

Từ những kết quả đã đạt được và các hạn chế đã nhận diện, nhóm đề xuất các hướng phát triển sau cho dự án SmartNutri:

- **Mở rộng cơ sở dữ liệu món ăn Việt:** Bổ sung thêm các món ăn từ nhiều vùng miền, bao gồm cả món ăn đường phố, món ăn gia đình và món ăn nhà hàng. Mỗi món cần được ghi nhận với thông tin dinh dưỡng đã được đối chiếu với nguồn tham khảo đáng tin cậy (Viện Dinh dưỡng Quốc gia, USDA, hoặc bảng thành phần thực phẩm Việt Nam). Về dài hạn,

có thể xem xét cơ chế để người dùng hoặc chuyên gia đóng góp và xác thực dữ liệu thực phẩm.

- **Cải thiện độ chính xác của AI:** Nghiên cứu fine-tune Gemini hoặc các mô hình thị giác khác trên tập dữ liệu ảnh món ăn Việt Nam có gán nhãn. Thiết kế prompt chi tiết hơn, bao gồm các đặc trưng nhận diện theo vùng miền và cách chế biến. Bổ sung cơ chế kiểm tra chéo giữa kết quả AI và thông tin dinh dưỡng từ cơ sở dữ liệu thực phẩm để tăng độ nhất quán.
- **Tích hợp thanh toán Google Play Billing:** Triển khai tính năng thanh toán in-app thông qua Google Play Billing để người dùng có thể tự đăng ký và gia hạn gói Premium. Hệ thống cần xử lý đồng bộ trạng thái subscription giữa Google Play và Firestore để đảm bảo quyền lợi người dùng được cập nhật chính xác.
- **Thêm tính năng nhắc nhở bữa ăn:** Phát triển tính năng gửi thông báo nhắc nhở người dùng ghi nhật ký bữa ăn vào các khung giờ chính trong ngày. Có thể kết hợp với dữ liệu thông kê để đưa ra nhắc nhở thông minh — ví dụ: nhắc người dùng ăn nhẹ nếu lượng calo trong ngày còn thấp hơn nhiều so với mục tiêu.
- **Phát hành ứng dụng lên Google Play Store:** Hoàn thiện các bước chuẩn bị để xuất bản ứng dụng lên Google Play, bao gồm: ký APK/AAB với keystore chính thức, thiết lập Firebase project production riêng biệt với môi trường phát triển, tối ưu kích thước APK, và chuẩn bị nội dung cho trang đăng ký trên Play Console (mô tả, ảnh chụp màn hình, chính sách bảo mật).

Dự án SmartNutri đã đặt được nền tảng vững chắc cho một ứng dụng theo dõi dinh dưỡng thông minh dành cho người Việt. Với các cải tiến và mở rộng theo định hướng trên, ứng dụng có tiềm năng trở thành một công cụ hữu ích, góp phần nâng cao nhận thức và thói quen dinh dưỡng lành mạnh trong cộng đồng.

Tài liệu tham khảo

- [1] Google LLC. “Firebase Authentication Documentation,” **urlseen 15 may 2026. url:** <https://firebase.google.com/docs/auth>
- [2] Google LLC. “Cloud Firestore Documentation,” **urlseen 15 may 2026. url:** <https://firebase.google.com/docs/firestore>
- [3] Google LLC. “Cloud Storage for Firebase Documentation,” **urlseen 15 may 2026. url:** <https://firebase.google.com/docs/storage>
- [4] Google LLC. “Cloud Functions for Firebase Documentation,” **urlseen 15 may 2026. url:** <https://firebase.google.com/docs/functions>
- [5] Google LLC. “Gemini API Documentation,” **urlseen 15 may 2026. url:** <https://ai.google.dev/gemini-api/docs>
- [6] Google LLC. “Flutter Documentation,” **urlseen 15 may 2026. url:** <https://docs.flutter.dev>
- [7] Google LLC. “Dart Programming Language Documentation,” **urlseen 15 may 2026. url:** <https://dart.dev/guides>
- [8] Meta Platforms, Inc. “React Documentation,” **urlseen 15 may 2026. url:** <https://react.dev/docs>
- [9] Microsoft Corporation. “TypeScript Documentation,” **urlseen 15 may 2026. url:** <https://www.typescriptlang.org/docs>
- [10] Evan You and Vite Contributors. “Vite Documentation,” **urlseen 15 may 2026. url:** <https://vitejs.dev/guide>